27考研408模拟卷1（解析）

勘误：

- 第16题选B，解析正确，答案打错了，把C改成B
- 代码题41题题目示例改成，结果：4321+321=4642。(十进制)。更加方便理解题意。解析代码已修正
- 第三题，CD选项都加上M，如C选项：$M-(rear-front+M)\%M$
- 42题，插入74，答案图片中得叶子节点关键字72，73改成73，74
- 第7题，D选项没出好，Floyd最好还是结合有向图，这题原本是有向图，为了结合生成树考点把有向图改成了无向图，忘记改D选项了，现增加一幅有向图，答案仍选B。
- 第31题，从系统缓冲区读入一个数据块到用户工作区的时间为40μs，CPU对用户工作区的一个数据块处理分析时间为30μs，跟图片一致
- 44题代码应添加注释
- 47题第二个ip第一个字节从43→45（不影响做题）

选择答案速查：

1-5：ACDCD 6-10：CBCDA 11-15:BCCAC 16-20:BDDBA
21-25:DABDB 26-30:ACCBD 31-35:ACCBC 36-40：ABDBC

41. 首先题意是小端存储，下列方法均有错误：
    - 看成大端，三次reverse，最多给3分
    - 简单的把结点值相加，比如567+234（小端），是不是得到7 9 （11）？没考虑进位要扣分，扣分不超过3分
    - 没考虑最后的高位进位，即没添加新节点，要扣分，扣分2分
    - 如果新增一个链表存储结果，扣1分，不符合题目要求的最小空间复杂度（参考答案给的就是开辟新的链表，要扣分）
    - 如果转换成数组，进行计算，再转回链表，空间复杂度On，扣3分
    - 如果把大数转成longlong或其他类型，进行加法，再存进去，因为大数加法本身就是为了防止超出基本类型数据范围而设计的，扣9分，

    （1）算法设计思想：

    ①从链表头部（十进制低位）进行加法，sum为当前节点的和，超过10的话，同步修改L1和L2的当前结点的值为sum%10，用t存放进位信息

    ②当某个链表遍历结束后，应继续使用较长的链表存放结果，实现O（1）空间复杂度，运算时要加上低位的进位

    ③若两个链表都遍历完，仍有进位，需要在较长链表的末尾新增一个结点保存进位

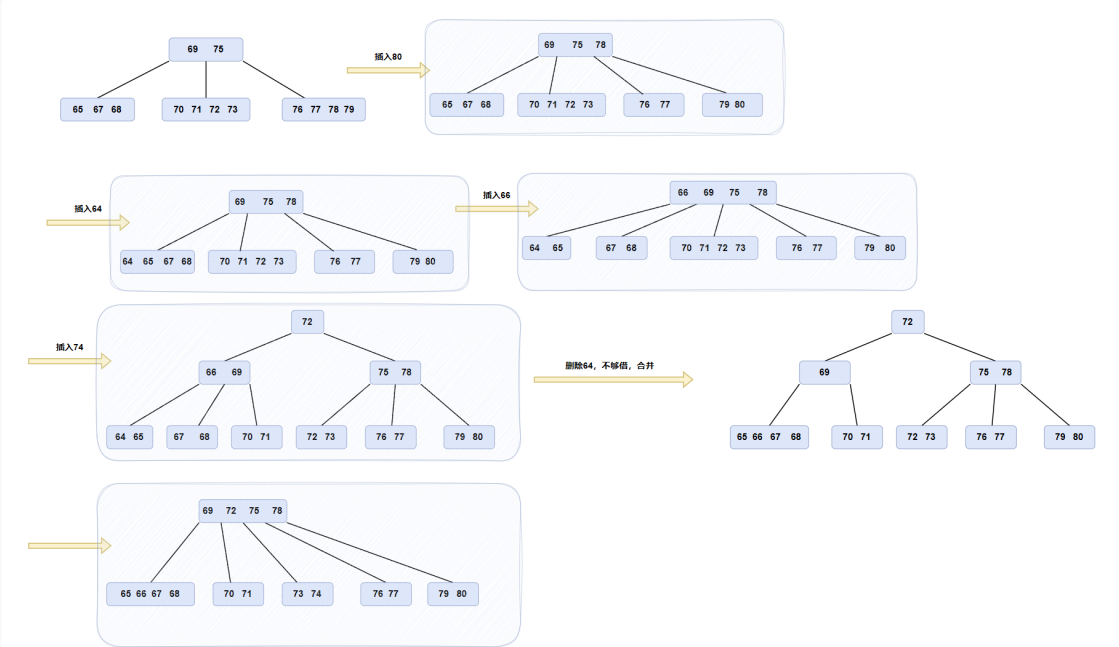
    ④返回较长链表的首元结点的指针

    (2)

```
ListNode* bignumAdd(ListNode* l1, ListNode* l2) {
        ListNode *p = l1, *q = l2, *pre_p = NULL, *pre_q = NULL; //pre_p和pre_q记录p,
q的前驱，使最后停留在尾结点，方便最后的进位的插入结点操作
        int t = 0; //记录进位信息
        int len1 = 0, len2 = 0; //记录l1和l2的链表长度
        while (p || q){
            int sum = 0, p_val= 0, q_val = 0;     //计算当前位数的和
            if (p) p_val = p->val;
            if (q) q_val = q->val;
            sum = p_val + q_val + t;
            if (p) p->val = sum % 10;
            if (q) q->val = sum % 10;
            t = sum / 10;
            if (p) {
                pre_p = p;
                p = p->next;
                len1 ++ ;
            }
            if (q) {
```

```
                pre_q = q;
                q = q->next;
                len2 ++ ;
            }
        }
        if (t) {    //处理最后的进位
            if (len1 > len2) pre_p->next = new ListNode(t);
            else pre_q ->next = new ListNode(t);
        }
        return len1 > len2 ? l1 : l2;
```

42. 【解析】插入80（2分）、64（1分），66（2分），74（2分），删除64（3分）



43. （1）二路组相联（组相联不给分）（1分）

（2）$Hit = C_1V_1 + C_2V_2$（2分）

（3）X：三态门（1分），Z：多路选择器（1分）。Z的功能是通过Hit信号控制数据来自Cache还是主存，若Hit为1表示cache命中，CPU从cache中取数据，反之从主存取数据。（2分）

（4）8/189（3分）

（5）4路组相联映射。（3分）19bit。（1分）

44. （1）200ps（1分）

（2）13（2分）

（3）前49次循环每次需要10个时钟周期（2分），最后一次13个时钟周期（1分）。（没说明最后一次循环13个时钟周期只给1分）

（4）①（2分）；403个时钟周期。（2分）

45. （1）FIFO缺页率为12/15 = 4/5（2分）；LRU缺页率为9/15=3/5。（2分）

（2）缺页9次；（2分）增加了修改位（1分）；在访问位同为0的情况下优先置换修改位为0的页面，因为未修改过的页面可以直接淘汰而不用写回磁盘，尽管增加了扫描次数（在内存），但是**减少了I/O次数**。（2分，言之有理即可）

46. （1）（10 + 128 + 128 * 128 + 128 * 128 * 128） * 512B（2分）

（2）12个。10个直接地址，1个一次间接地址，1个二次间接地址。（2分）

（3）最少2次（1分），最多5次。（1分）

47. （1）①：06H，②：00 00H，④：84 6B 41 C6H。（各1分）

（2）③：1D EFH。（2分）

（3）主机H发送：2025B（1分），接收0B（1分），序号为E0599FF0H（1分），确认号为846B49B0H（1分）。

一、单项选择题：01 ~ 40小题，每小题2分，共80分。下列每题给出的四个选项中，只有一个选项是最符合题目要求的。

1. 以下是经典汉诺塔问题的伪代码，在调用 `hanoi(n, 'A', 'C', 'B');` 时，时间复杂度是（）。

```
void hanoi(int n, char from_rod, char to_rod, char aux_rod) {
    if (n == 1) {
        只有一个盘子时，直接移动到目标柱子
        return;
    }
    hanoi(n-1, from_rod, aux_rod, to_rod);
    将最大的盘子（第n个）从起始柱子移动到目标柱子
    hanoi(n-1, aux_rod, to_rod, from_rod);
}
```

A. O（2^n）

B. O（n）

C. O（n^2）

D. O（nlogn）

> 答案：A。【解析】此题也可考察盘子移动次数，难度会更高。设n个盘子的汉诺塔问题移动次数为S(n)，推导如下：
>
> ①n=1，只需要将这个盘子从源塔移动到目标塔，S(1) = 1；
>
> ② 分析代码：
>
> - `hanoi(n-1, from_rod, aux_rod, to_rod);` //将上面的n-1个盘子从起始柱子通过目标柱子移动到辅助柱子 这个递归的移动次数为S(n-1)
> - 将最大的盘子（第n个）从起始柱子移动到目标柱子 由①可知，移动1次即可
> - `hanoi(n-1, aux_rod, to_rod, from_rod);`//将那n-1个盘子从辅助柱子通过起始柱子移动到目标柱子 这个递归的移动次数也为S(n-1)
>
> ③$S(n) = 2S(n-1) + 1 \rightarrow S(n) + 1 = 2(S(n-1) + 1) = 2^2(S(n-2) + 1) = \ldots$
> $= 2^{n-1}(S(1) + 1) = 2^n \rightarrow S(n) = 2^n - 1$
>
> 综上，盘子的移动次数为2^n-1，即时间复杂度为O（2^n），答案选A。

2. 一个容量为3的栈的进栈序列为1,2,3,4，那么有（）个不同的出栈序列?

A. 15　B. 14　C. 13　D. 12

> 答案：C 【解析】若栈容量为4，则不同出栈序列数为卡特兰数$\frac{C_{2n}^n}{n+1}$,n=4时经计算为14。但题目容量限制为3，只有4,3,2,1出栈序列需要容量为4，无法实现，故不同的出栈序列数量为14-1=13。

3. 循环队列放在一维数组$A[0, \ldots, M-1]$中，$front$指向队头元素，$rear$指向队尾元素的后一个位置。数组大小不可扩展，并且不考虑额外开销的情况下，该**队列**的剩余可用空间为（）

A. $M - (rear - front)$

B. $M - (rear - front)\%(M-1)$

C. $M - (rear - front + M)\%M$

D. $M - 1 - (rear - front + M)\%M$

> 答案：D。【解析】循环队列由固定大小M的数组A实现，队列必须空出一个单元来区别队空和队满的判断。也可以用其他方法，但是都需要标志信息的额外开销，比如用count来记录队列元素个数，使用标记位（入队置true，出队置false，若rear和front无法区分队满还是队空，用标记位true表示满，false表示空）等，但都需要额外的开销，所以必须牺牲一个队列空间，故队满情况下元素个数为M-1。$(rear - front)\%M$为当前队列的元素个数，因此答案选D。

4. 哪种线索二叉树需要（非递归）栈完成各自的遍历？（）

Ⅰ. 前序线索二叉树　Ⅱ. 中序线索二叉树　Ⅲ. 后序线索二叉树

A. Ⅰ、Ⅲ

B. Ⅰ

C. Ⅲ

D. Ⅰ、Ⅱ、Ⅲ

答案：C。【解析】能够在各自的线索二叉树上找到当前节点对应的后继结点，才能一直往后继续完成遍历。只有后序遍历不能找到后继，所以需要借助栈来完成后序线索二叉树的后序遍历。

	中序线索二叉树	先序线索二叉树	后序线索二叉树
找前驱	✓	✗	✓
找后继	✓	✓	✗

5. 一棵二叉排序树的层序遍历序列是30,20,39,10,23,35,42,15,25。这棵二叉树转换成的树/森林的后根遍历为()

A. 10,15,25,23,20,35,42,39,30

B. 30,20,10,15,23,25,39,35,42

C. 15,10,23,25,20,35,42,39,30

D. 10,15,20,23,25,30,35,39,42

答案：D。【解析】二叉排序树的中序遍历为有序序列 10,15,20,23,25,30,35,39,42。二叉树转换的森林的后根遍历对应二叉树的中序遍历，故答案为D。也可尝试层序+中序构造一棵二叉树可知B为先序，C为后序，不过稍显复杂。

6. 下列关于图的说法，正确的有（）

Ⅰ. 一个有向图的顶点数为n（n>1），边数为m，若该有向图强连通，则n≤m

Ⅱ. 顶点数为n的有向图，任意情况下强连通，至少需要$2C_{n-1}^2+1$条边

Ⅲ. 结点数为n的有向树，有向边一定为n-1

Ⅳ. 顶点数为n的无向图若是连通图，至少需要C_{n-1}^2+1条边

A. Ⅰ、Ⅱ、Ⅲ

B. Ⅰ、Ⅲ、Ⅳ

C. Ⅰ、Ⅲ

D. Ⅰ、Ⅱ、Ⅲ、Ⅳ

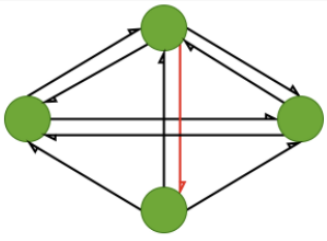
答案：C。【解析】此题考察图的概念和相关性质。

Ⅰ正确，n个顶点的有向图强连通，至少需要n条边（构成一个环），所以m≥n。

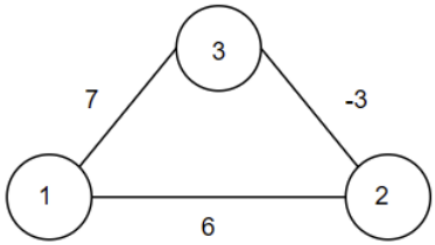
Ⅱ错误，要保证任意性，n-1个结点要保证多浪费尽可能多的边数，也就是完全有向图，这时候边数是$2C_{n-1}^2$。最后一个顶点向n-1个顶点各发出一条出边，这样保证了$n-1$个结点是强连通的，而最后一个顶点不连通，这时候是最糟糕的情况（消耗的边数最多）。此时任加一条边，都会构成强连通图。经以上分析，可以得到公式：$n+2C_{n-1}^2$，$n=4$时图如下。

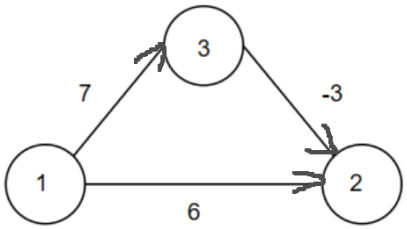
Ⅲ正确，有向树顶点与边的关系类似生成树，都是边数m=n-1。

Ⅳ错误，至少需要n-1条边（一条链），如果表述改成"在任意情况下连通，至少需要C_{n-1}^2+1条边"，则是正确的。



7. 根据下图，关于Dijkstra、Prim、Floyd算法说法正确的是（）





A. 用Dijkstra算法求上面右图的结点1单源最短路径，到结点2和结点3的最短路径长度分别为4和7

B. 用Prim算法求左图最小生成树，从结点1出发，得到的最小生成树保留的边权值为-3和6

C. Dijkstra和Prim算法的思想都是基于贪心，都是从当前轮选出的结点作为中间结点去更新未处理结点的path数组，且更新的策略一样

D. 用Floyd算法求右图的最短路径，经过一轮计算后，得到的矩阵 $A^0 = \begin{bmatrix} 0 & 4 & 7 \\ \infty & 0 & \infty \\ \infty & -3 & 0 \end{bmatrix}$

答案：B。【解析】A错误， Dijkstra算法不能求上图的结点1单源最短路径，从结点1出发，先选择结点2，最短路径长度确定为6，然后选择结点3，最短路径长度为7。说明了 **Dijkstra算法不能求解带负权图的单源最短路路径。**

B正确，Prim算法①从结点1出发，结点1的相关边的权值为6、7，所以优先选6，即结点2加入。②以1，2作为一个整体，找出离这个整体的最短路径连接的未处理的结点，-3是所有的边中最短的，所以结点3加入，至此所有结点都已在生成树里面，B正确。

C错误，更新的策略并不一致，这也是Prim和Dijkstra的区别，设当前选定的结点为cur，图的边权用w数组存储，要更新的未处理的结点为i，

$Prim$算法：$path[i] = min(path[i], w[i])$

$Dijkstra$算法：$path[i] = min(path[i], w[i] + path[cur])$

注：Prim是以整个集合为源，Dijkstra是以单个结点如本题的结点0为源，需要加上path[cur]来存放0→cur的最短路径长度信息。

D错误，这是最后一轮计算得到矩阵A^2。因为唯一会更新的是A[1] [2] = A[1] [3] + A[3] [2]，以结点3为中间结点，那就是在第三轮计算得到的。

通过本题，希望大家去复习和整理最小生成树，最短路径算法的相关考点，对照代码去理解算法的思想，真题考到了不至于整个空着。

8. 在含有 15个结点的平衡二叉树上，查找关键字为 36(存在该结点)的叶子结点，则依次比较的关键字可能是( )

A. 39，64，36

B. 37，38，47，36

C. 12，63，35，45，36

D. 25，37，28，31，35，36

答案：C。【解析】设 N_h 表示深度为h的平衡二叉树中含有的最少结点数，有$N_h = N_{h-1} + N_{h-2} + 1$
$N_0 = 0, \mathrm{N}_1 = 1, \mathrm{N}_2 = 2, \ldots, \mathrm{N}_\mathrm{h} = \mathrm{N}_{\mathrm{h}-1} + \mathrm{N}_{\mathrm{h}-2} + 1, \mathrm{N}_3 = 4, \mathrm{N}_4 = 7,$
$N_5 = 12, N_6 = 20 > 15$。也就是说，高度为6的平衡二叉树最少有20个结点，因此15个结点的平衡二叉树的高度最高为5，由此排除选项D。平衡二叉树也是一棵排序二叉树，对于A选项，64>39而36<39，因此不符合二叉排序树，同理B中，38、47>37而36<37，也不符合二叉排序树的规则，答案选C。

拓展：对于结点数为n的平衡二叉树，叶子结点的最大高度和最小高度的差是多少?

那么对于结点数为n的avl，可以推出最大高度，也就是叶子结点所在的最大层号。那么怎么求最小层数呢？由于$N_h = N_{h-1} + N_{h-2} + 1$是递归定义的，可以由n求得最大高度h，设为$AVL_h$。然后由递归的性质，$AVL_h$左右两子树分别$AVL_{h-1}, AVL_{h-2}$，假设$AVL_{h-2}$的叶子结点最大和最小高度之差为$P_{h-2}$，那么$AVL_{h-1}$的叶子结点最大高度是h-1，$AVL_{h-1}$和$AVL_{h-2}$分局根节点两侧的情况下，整棵$AVL_h$的叶子节点高度之差一定是由$AVL_{h-1}$的最高叶子结点和$AVL_{h-2}$的最小叶子结点形成的高度差，$AVL_{h-1}$的最高叶子结点是h-1，$AVL_{h-2}$的最高叶子结点是h-2，所以$P_h = P_{h-2} + 1$，可以根据和初值$P_2 = 0$，$P_3 = 1$这个来求得

9. 设Hash表长14，Hash函数为$H(key) = key \ mod \ 11$，依次将15，38，61，84，49添加到表中，用平方探测法解决冲突，则49放入的位置是（）

   A. 8

   B. 3

   C. 5

   D. 9

答案：D。【解析】前四个关键词都未发生冲突，添加49进入后，d = 49 mod 11 = 5，发生冲突，然后按照$d+1^2, d-1^2, d+2^2, d-2^2 \ldots$进行探查，当d+$2^2$=9处有空位置，因此本题答案为D。

0	1	2	3	4	5	6	7	8	9	10
				15	38	61	84			

10. 将关键字7,5,1,4,6,8,9依次插入初始为空的小根堆H,得到的H是( )。

    A.1,4,5,7,6,8,9

    B.1,4,7,5,6,8,9

    C.1,4,5,6,7,8,9

    D.1,5,4,6,8,9,7

答案：A。【解析】注意区分题目的信息，如果题目说的是将关键字进行堆排序或者构建一个小根堆，那么需要进行建堆（调整）的操作。但此题已经是一个小根堆，需要一个个插进去，每次将关键字查到堆尾，向上调整。然后下一次插入，直到所有的关键字都插入到小根堆为止。过程如下：

已插入的关键字	插入调整后的小根堆
7	7
7，5	5，7
7，5，1	1，7，5
7，5，1，4	1，4，5，7
7，5，1，4，6	1，4，5，7，6
7，5，1，4，6，8	1，4，5，7，6，8
7，5，1，4，6，8，9	1，4，5，7，6，8，9

11. 在对大量数据进行外部排序时，以下()策略不能有效降低磁盘 IO 操作次数。

    A.选择置换排序

    B.败者树

    C.最佳归并树

    D.适当增大归并路数

答案：B。【解析】I/O次数为形成初始归并段的内部排序和归并排序的磁盘I/O次数和归并排序的I/O次数。归并排序趟数$S = log_k r$，其中k为归并路数，r为初始归并段数，

归并排序I/O次数$= S *$（记录总数/单个磁盘块所能容纳记录数）$* 2$，* 2是因为读入和写出各一次。

所以要减少I/O就要减少S，$S = log_k r$，减少S可以增大k或者减少r，A选项选择置换排序是减少初始归并段数r，但此时每个初始归并段大小不一样，所以需要C选项最佳归并树得到最小的wpl（I/O次数）。D选项是增大k，同样可以减小S，但由于内存缓冲区资源有限k，k路归并需要至少k个缓冲区，所以k不可无限增大。B选项是为了减少归并排序的比较次数，使得每次从k个缓冲区的首元素选出最小的数，比较次数k-1降为$log_2 k$，与I/O次数无关，故此题选B。

12. 某基准程序在主频400MHz的处理器上运行，其目标代码有100000条指令，由如下各类指令及时钟周期计数混合而成：

指令类型	指令条数	CPI
整数运算	45000	1
数据传送	32000	2
浮点数运算	15000	3
控制传送	8000	2

该程序的执行时间和有效MIPS分别约是（）

A：250μs、400　　B：400μs、250　　C:425μs、235　　D:500μs、200

答案：C。【解析】首先根据各类型指令的条数可算出各类型指令的占比，整数运算占比0.45，数据传送占0.32，浮点运算占0.15，控制传送占0.08。平均CPI=1×0.45+2×0.32+3×0.15+2×0.08=1.7，则MIPS=主频/CPI=400/1.7≈235，程序执行时间=1.7×100000/400M=425μs，因此选C。注意，算MIPS时，主频默认单位为MHz且不写出。

13. 假设$1e10 = 1.00101010000001011111001 \times 2^{33}$，附加位为2位（又叫保留位，用于保留移出的尾数，便于后续舍入），并采用就近舍入方式。在C语言程序中，下列程序段中执行后，a的值为()

```
float a=512+1e10;
a=a-1e10;
```

A. 0

B. 512

C. 1024

D. 256

答案：C。【解析】浮点数运算步骤为：对阶、尾数运算、尾数规格化、舍入。

a=512+1e10，1e10=1. *** 1× 2，512=1×2^9。浮点数加法第一步为对阶，小阶向大阶对齐。2^9对阶到2^{33}时，需要右规24位，隐藏位的1刚好为2^{-24}，尾数只有23位，所以应该放入附加位第一位中，接着进行尾数加法、规划化等步骤，最后通过附加位舍入，由于两位附加位为10。尾数相加运算结果最后一位也是1，所以就近舍入方式下，舍入后尾数最低位+1.因此512舍入后变成了1024，即a变成了1e10+1024，所以a=a-1e10后最终结果a=1024。注意就近舍入方式是4舍6入，5取偶数，由于附加位是10，属于中间数“5”，所以要看尾数最后一位是奇数还是偶数，由题目信息最后一位是1，是奇数，所以要进1，使最后一位变成0，得偶数。如果最后一位是0，则不进位，直接舍弃，保持偶数0的状态。

IEEE 754 还提供向正无穷舍入、向负无穷舍入、向0舍入（截断法），有兴趣了解的可以自行查阅资料。

思考：这题若是a=511+1e10，结果是多少？（0）**这题也间接说明了阶码之差要大于24才能直接取较大的数。**

14. 在按字节编址，采用小端存储方式的32位计算机中，按边界对齐方式为以下的C语言结构体变量a分配空间

```
struct Inner{
    char a;
    short b;
    int c;
    char d;
}
struct Outer{
    short e;
    Inner inner;
    short f;
    int m;
}
Outer A;
```

若A的首地址为2025 BC00H，a的成员变量m的机器数为1234 5678H,则其中56H所在的存储单元地址为（）

A:2025 BC15H

B:2025 BC16H

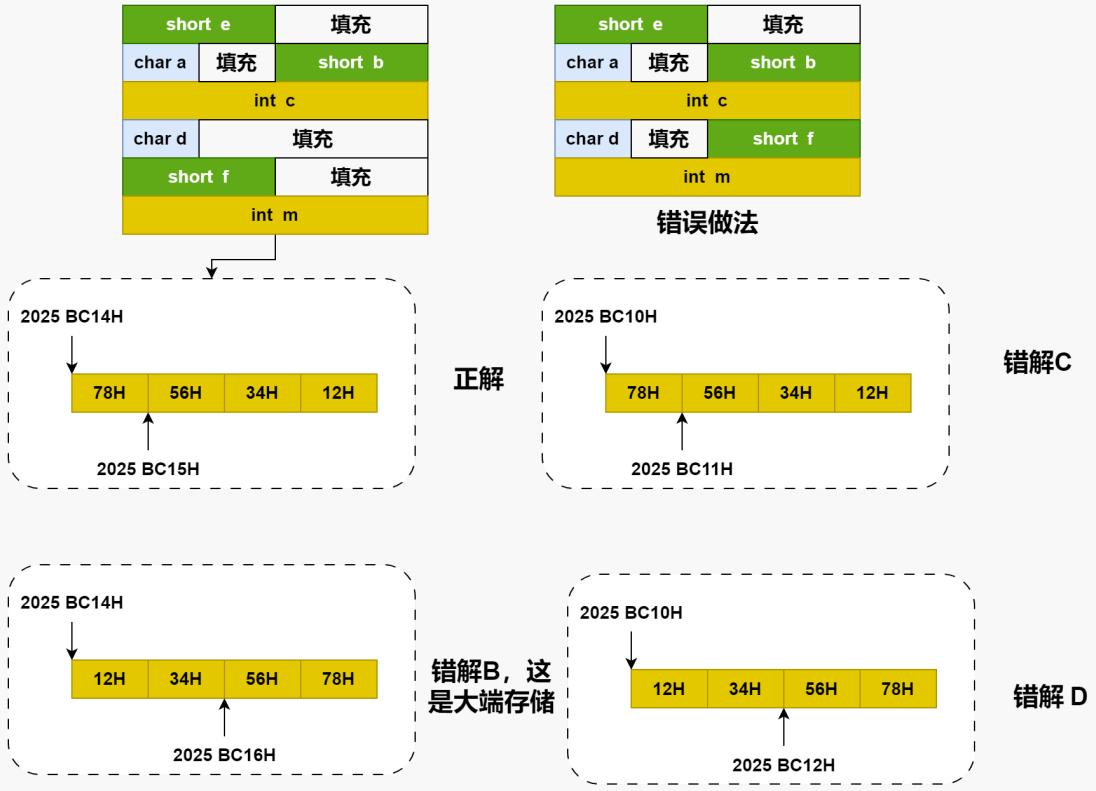
C:2025 BC10H

D:2025 BC12H

答案：A。【解析】结构体对齐总结：

- 成员变量为0的结构体占1B
- 结构体本身对齐按最大成员变量的宽度对齐
- 内部成员本身按自己的数据类型对齐

题目的具体分布如下图所示：



15. Booth 算法是一种高效的补码一位乘法算法，它利用对乘数和被乘数的特定编码来减少所需的加法和减法操作次数。那么设x=-0.1101，y=0.1011，在利用 Booth 算法求x·y时，部分积加[-x]补的次数为()。

A.0　B.1　C.2　D.3

答案：C。【解析】

这题首先在y后面添0（y_{-1}），然后从后往前看，每次看两位y_0y-1，只有出现**‘10’**才+[-x]补，可知出现两次。

补码一位乘法(Booth算法)运算规则如下。
（1）符号位参与运算，运算的数均以补码表示。
（2）被乘数一般取双符号位参与运算，部分积取双符号位，初值为0，乘法取单符号位。

（3）乘数末位增设附加位 y_{n+1}，初值为 0。

（4）根据(y_0，y_{-1})的取值来决定操作，Booth 算法规则详细见下:

- $y_0y_{-1}=01$，部分积+$[X_{补}]$
- $y_0y_{-1}=10$，部分积+$[-X_{补}]$
- $y_0y_{-1}=00$或11，部分积+0

16. 某计算机主存按字节编址，由4个128M×8位的DRAM芯片采用交叉编址方式构成，与宽度为32位的存储器总线相连，主存每次最多读写32位数据。考虑到在减少地址线数量的同时尽可能减少刷新开销，则下列说法错误的是（）

A. DRAM芯片的数据引脚数为8

B. DRAM芯片有地址引脚数为13

C. DRAM芯片的行缓存大小为16KB

D. CPU访问该主存数据采用同时启动的方式

答案：B。【解析】A正确，128M×8位中的8即为数据引脚数量

B错误，DRAM芯片128M=2^{27}，采用地址复用技术，行列地址分两次送，地址线要取行列地址线中的较大者，要使地址线数量尽可能少，行列地址位数之差要尽可能靠近。同时要尽可能减少刷新开销，DRAM按行刷新，因此行地址要尽可能小，因此27分成行列地址位数，方案应该为13，14。因此地址线$\lceil 27/2 \rceil = 14$根

C正确，行缓存为存储阵列中一行的数据大小，一共2^{14}×8bit=16KB。

D正确，存储器总线宽度为32位，与存储字长8bit是倍数关系，所以是同时启动。（相等关系是轮流启动）

17. `jge addr` 表示条件跳转指令，`jmp addr` 为无条件跳转指令，addr为目标地址，按字节编址，对于如下的C语言循环语句及其机器级表示的代码，若jge的机器指令实现采用相对寻址，那么其机器指令里的地址码（8位补码）为（），addr2地址为（）

```
00E1176D mov [count], 0          int count=0;
                                 while(count<10)
00E11774 cmp [count], 10
00E11778 jge addr1
                                 {
00E1177A mov eax, [count]
00E1177D add eax, 1               count ++ ;
00E11780 mov [count], eax
00E11783 jmp addr2
                                 }
00E11785 ...     //while循环语句块之后的指令
```

A. 0DH，00E11785H

B. 0DH，00E11774H

C. 0BH，00E11785H

D. 0BH，00E11774H

答案：D。【解析】首先要确定 `addr1` ，由C代码 `while(count<10)` ，count小于10，执行循环体，count≥10，结束循环， `jge` 为跳转指令， `ge:greater equal` ，满足≥进行跳转，即跳到循环结束之后的指令，由上述代码可知addr1=00E11785H，对于相对寻址，是在PC+'1'后进行偏移，即下一条指令地址00E1177AH。于是8位偏移的补码为00E11785H-00E1177AH=0BH。循环内执行完，要重新进行比较，所以无条件跳转指令的目的地址是cmp指令，即00E11774H，答案选D。

18. 对于微程序控制器，下列说法正确的是（）

A. 一条机器指令对应一个微程序，该条机器指令的执行中首条微指令的地址由微地址形成部件给出

B. 一个微程序对应若干条微指令，垂直型微指令较长，且一条微指令只能发出一个微命令

C. 对于字段直接编码方式的微指令，互斥性微命令分居不同字段，各字段送往同一个译码器进行译码

D. 字段间接编码方式的主要作用是缩短微指令字长

答案：D。【解析】A错误，机器指令的执行，首条微指令应该和取指相关，该条微指令的地址由硬件自动生成；取出指令后，将指令的操作码送往微地址形成部件得到机器指令对应的微程序继续执行。B错误，垂直型微指令的特点是微指令较短，但是一条微指令只能发出一个微命令，导致由垂直型微指令编写的微程序很长。C错误，不同字段单独译码。故答案选D。

19. 下列关于单周期数据通路的叙述中，正确的是（）

I. 单周期数据通路中PC寄存器写使能信号一直有效

II. 单周期处理器中取指令操作部件中，取指令操作不能和下条指令地址的计算操作同时进行

III. 在一条指令的执行过程中，单周期处理器的每个控制信号取值一直不变

IV.一般来说，单周期处理器的时钟周期比多周期处理器的时钟周期长

V. 单周期数据通路中，任何资源都不能被重复使用，应该使用专用数据通路

VI. 单周期处理器中，指令从存储器读入IR，然后从IR取出执行

A. I、II、IV、VI

B. III、IV、V

C. II、IV、VI

D. I、II、III、IV、V、VI

B【解析】

I.错误. 因为是单周期处理器,每个时钟周期执行一条指令,所以每来一个时钟,PC 的值都会被更新一次,因而,PC 无须“写使能”信号控制。

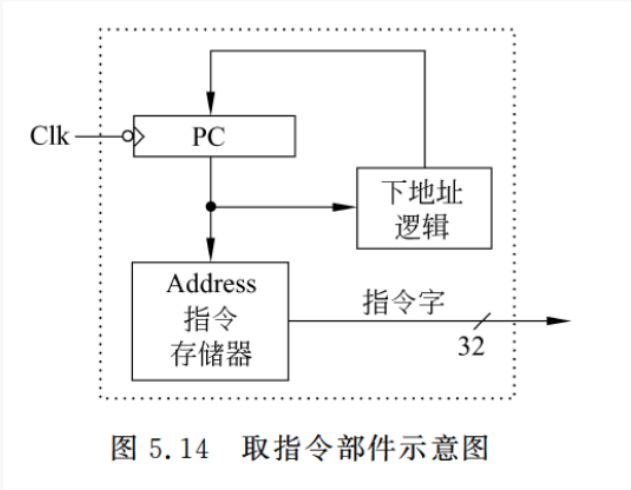


图 5.14　取指令部件示意图

单周期处理器取指部件：假定指令专门存放在指令存储器中，它只有读操作，读指令操作可看成是组合逻辑操作，因此无须控制信号的控制，只要给出指令地址，经过一定的“取数时间”后，指令被送出。且有专门的下地址逻辑来计算下条指令的地址，然后送 PC。**因为是单周期处理器,每个时钟周期执行一条指令，所以每来一个时钟，PC 的值都会被更新一次,因而，PC 无须“写使能”信号控制。**下地址逻辑中,要区分是顺序执行还是转移执行。若是顺序执行,则执行 PC+"1"，若是转移执行，则要根据当前指令是分支指令还是跳转指令来计算转移目标地址。

II.错误。因为该时钟周期结束后，要执行下一条指令，在执行下一条指令之前，必须在本条指令中得到下一条指令的地址，所以必须在同一个时钟周期内同时完成。也可参考上图的取指部件

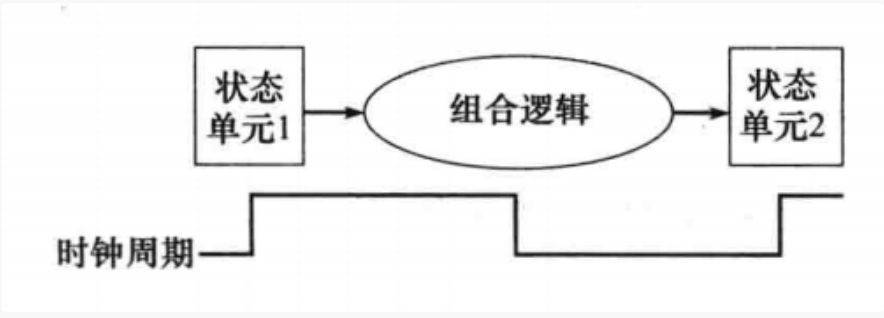
III.正确，单周期处理器的控制信号总是在下一个时钟周期开始时，从指令存储器中取出指令将操作码部分送到控制器后才改变，在指令执行过程中一直保持不变。

IV.正确，**单周期MIPS处理器每个时钟周期执行一条指令，CPI=1，看似性能很好，但由于该处理器的时钟周期对所有指令等长，因此只能以执行速度最慢的指令作为设计其时钟周期的依据。**

V.正确，**由于只能在一个时钟周期内完成指令的取出和执行操作，指令执行过程中数据通路的任何资源都不能被重复使用，**都应该是专用的数据通路，需要被多次使用的资源(如**加法器**)都需要设置多个;取指令和取操作数操作都需要访问存储器，因此**将指令和数据分别存放在指令存储器和数据存储器**中，以避免资源冲突。

VI.错误，**对于单周期数据通路，一条指令执行的结果总是在下一个时钟周期开始时写入状态元件，而在指令执行过程中没有任何状态元件被写入信息，否则一个指令周期不可能在一个时钟周期内完成。**指令寄存器是一种状态元件，显然一个时钟周期内无法同时完成对同一个状态元件的读写。

补充：**时钟信号长度的确定**：下图描述了一个组合逻辑单元及与其相连的两个状态单元。组合逻辑单元的操作在一 个时钟周期内完成，所有信号在一个时钟周期内从状态单元1经组合逻辑到达状态单元2，信号到达状态单元2 所需的时间决定了时钟周期的长度。



20. 关于高级流水线的说法中，正确的是（）

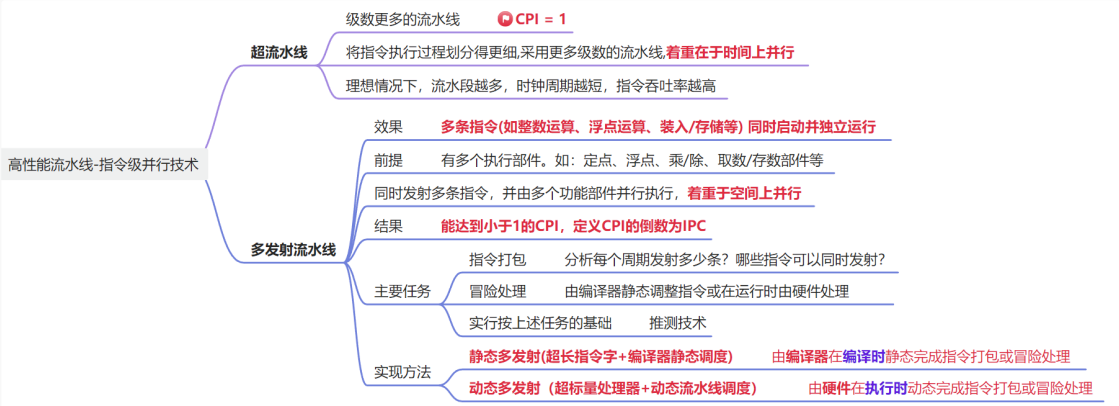
A. 超流水线采用更多级的流水线，着重于在时间上并行；多发射流水线同时发射多条指令，并且由多个功能部件并行执行，着重于空间上并行。

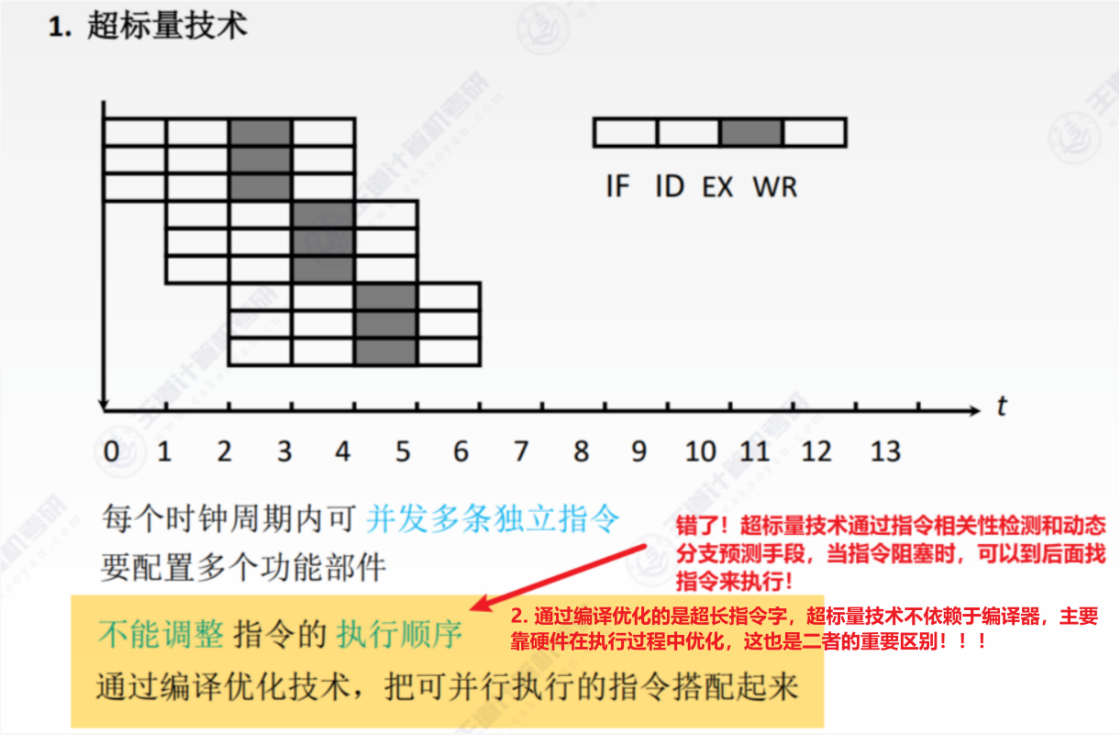
B. 关于多发射流水线的实现，由编译器在编译时完成指令打包或冒险处理的是动态多发射（超标量处理器），由硬件在执行时完成指令打包或冒险处理的是静态多发射（超长指令字）

C. 超流水线技术通过增加流水线级数来使更多的指令同时在流水线中重叠执行，将指令的执行过程划分为更多的级数，因此CPI<1

D. 超标量流水线技术不能调整指令的执行顺序，通过编译优化技术，把可并行执行的指令搭配起来一起发射。

A。【解析】：本题选A；B说反了，编译器编译时处理是静态多发射（超长指令字），硬件在执行时动态处理是动态多发射（超标量处理器）。C中超流水线将流水线进一步细分，时钟周期变小短，但CPI=1。D，超标量流水线技术可以调整指令的执行顺序，跟编译无关。





21. 以下关于CPU响应外部中断请求的叙述中，错误的是（）

A. 在中断响应周期，CPU将中断允许触发器清0，以使CPU关中断

B. 系统处于开中断时，只要CPU检测到中断信号就会立即响应

C. 在中断响应周期，CPU把取得的中断服务程序的入口地址送往PC

D. CPU允许中断且处于开中断状态，每条指令结束后，CPU都会转到“中断响应周期”进行中断相应处理

D。【解析】CPU在每条指令结束后检测中断请求信号，若检测到中断请求信号有效，则进入中断响应周期；若检测到中断请求信号无效，则不会进入中断响应周期。

22. 某计算机的CPU主频为500MHz，某外设接口有一个16位的数据缓冲器，响应的中断服务程序的执行时间为500个时钟周期，下列说法正确的是（）

A. 若该外设的最大数据传输速率为20kB/s，那么CPU用于该设备输入输出的时间占整个CPU时间的大约为1%

B. 若该外设的最大数据传输速率为20kB/s，那么CPU用于该设备输入输出的时间占整个CPU时间的大约为0.1%

C. 若该外设的最大数据传输速率为2MB/s，那么CPU用于该设备输入输出的时间占整个CPU时间的大约为10%

D. 若该外设的最大数据传输速率为2MB/s，那么CPU用于该设备输入输出的时间占整个CPU时间的大约为100%

答案：A。【解析】中断过程中，CPU用于设备输入输出的时间包括中断服务程序的开销和中断响应的开销，中断响应就是执行一条隐指令的过程（硬件实现），所以其开销远小于中断服务程序的时间。

外设的最大数据传输速率为20kB/s，一次中断完成16bit=2B数据的传输，那么中断请求的时间间隔为 $\frac{2}{20\times10^3}s=100\mu s$，对应的中断服务程序的执行时间为 $\frac{1}{500\times10^6}s=1\mu s$，所以占比大约为1%（考虑到中断响应时间，所以用大约），A正确。

同样的计算方式可以算得外设的最大数据传输速率为2MB/s时，CPU用于该设备输入输出的时间占整个CPU时间的大约为100%，但是这时候考虑中断响应时间，就不能用大约了，因为超过100%，该设备不适合中断方式进行数据传输了，D错误。



23. 对于微内核与宏内核操作系统设计的区别，下面说法正确的是（）

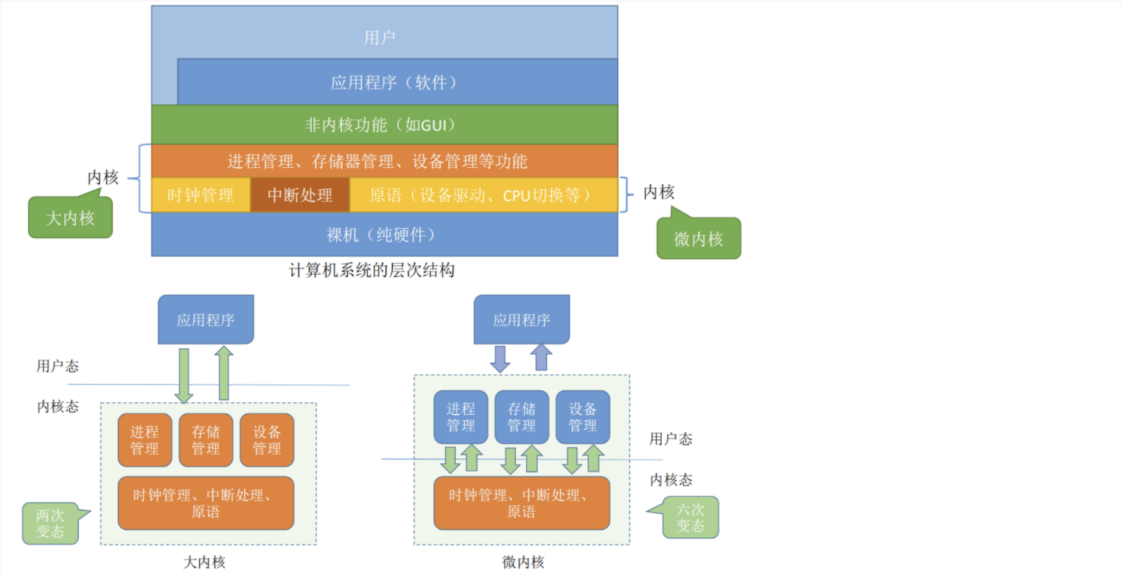
A. 宏内核设计下，所有内核功能模块都运行在内核态下，微内核设计下，所有内核功能模块都运行在用户态。

B. 宏内核设计下不同模块间通信通过过程调用，而微内核设计下，不同模块间通信不能直接通过过程调用。

C. 宏内核相较于微内核设计下，特权级切换更为频繁

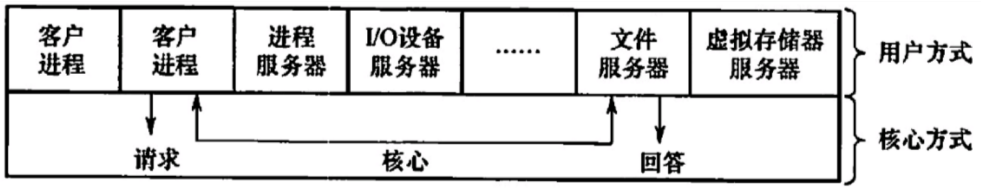
D. 相对于宏内核设计来说，微内核设计具有更好的安全性和性能。

答案：B。【解析】**本题选择B。**选项A错误，在微内核架构中，最基本的功能仍然运行在内核态，而将那些不需要在核心态下执行的功能移到用户态执行，从而降低内核的设计复杂性，因此A错误。



微内核不同模块运行在不同的地址模块，各功能模块之间通过内核提供的消息传递机制进行通信；而宏内核中不同模块运行在同一地址空间，因此可以直接过程调用，所以B选项正确。

由于微内核将机制与实现分离在不同特权级，所以需要**频繁地在核心态和用户态之间进行切换，操作系统的执行开销偏大**，因此C错误。微内核可靠性确实更好，因为只有微内核运行在内核态，其余模块都运行在用户态，**一个模块中的错误只会使这个模块崩溃，而不会使整个系统崩溃**，但是**微内核性能更差**，这个不仅由于需要频繁切换特权级，更多的是因为调用别的功能模块需要切换地址空间，因此D错误。



24. 下列关于系统启动过程，说法正确的是()

Ⅰ. BIOS实现硬件自检功能，并引出启动磁盘的第一个分区的第一个扇区主引导记录MBR

Ⅱ. 引导扇区的功能是加载操作系统的内核或其他必要的启动代码到内存中

Ⅲ. 操作系统内核加载完成后，会创建虚拟文件系统（VFS），然后挂载根文件系统并初始化其元数据

Ⅳ. 所有驱动程序的加载都在操作系统初始化中完成，在这个过程中注册中断向量表

A. Ⅰ、Ⅱ、Ⅲ、Ⅳ

B. Ⅰ、Ⅱ、Ⅲ

C. Ⅰ、Ⅱ

D. Ⅱ、Ⅲ

答案：D。【解析】Ⅰ错误，BIOS实现硬件自检功能，引出的MBR是磁盘的第一个扇区，并不属于任何分区；

Ⅱ正确，活动分区的引导扇区主要实现①初始化计算机硬件如设置数据段、堆栈段和堆栈指针等②加载操作系统③处理错误等功能。

Ⅲ正确，在内核加载完成后，操作系统会创建VFS并挂载根文件系统。挂载过程包括将根文件系统与某个存储设备关联起来，并将其挂载到根目录（/）下。在挂载根文件系统时，操作系统会读取并初始化根文件系统的元数据。元数据包括超级块、索引节点等，它们描述了文件系统的结构和内容。VFS在系统启动时由内核创建，并在系统关闭时销毁，只存在于内存中。

Ⅳ错误，并不是所有驱动程序的加载都在操作系统初始化中完成。写入内核的驱动程序（静态驱动程序）在启动内核时加载，其他的在系统初始化的时候由内核加载，支持即插即用的设备在插入时加载，不支持即插即用的驱动程序需要手动安装。驱动程序的加载过程中需要申请中断号并且注册中断向量表，这个说法是正确的。

25. 下列选项不属于进程调度的时机的是（）

A. 阻塞一个进程

B. 申请内核锁成功时

C. 中断返回之前

D. 终止一个进程

答案：B。【解析】进程调度是将CPU资源分配给一个用户进程。

A选项中，阻塞是进程自身的行为，进程加入阻塞队列后需要调度新的进程上CPU。

B选项中，内核锁属于内核数据结构，内核临界区不能发生进程调用，故答案选B。

C选项中，在每次系统调用和中断之后，标志位need_resched被例程ret_from_sys_call检查，如果标志被置位，那么调度程序也需要在此时被调用。

D选项中，当前进程被终止时，需要调度一个新的进程上处理机。

26. 假设在单处理机**两道程序**设计环境下，有4个作业，它们进入系统的时间如表所示，作业调度采用短作业优先调度算法，进程调度采用优先级抢占式调度算法。这里规定:优先数越小优先级越高。忽略其他辅助操作的占用时间。平均周转时间为（）

作业	进入系统时间	估计运行时间（分钟）	优先数
JOB1	10:00	40	5
JOB2	10:20	30	3
JOB3	10:30	50	4
JOB4	10:50	20	6

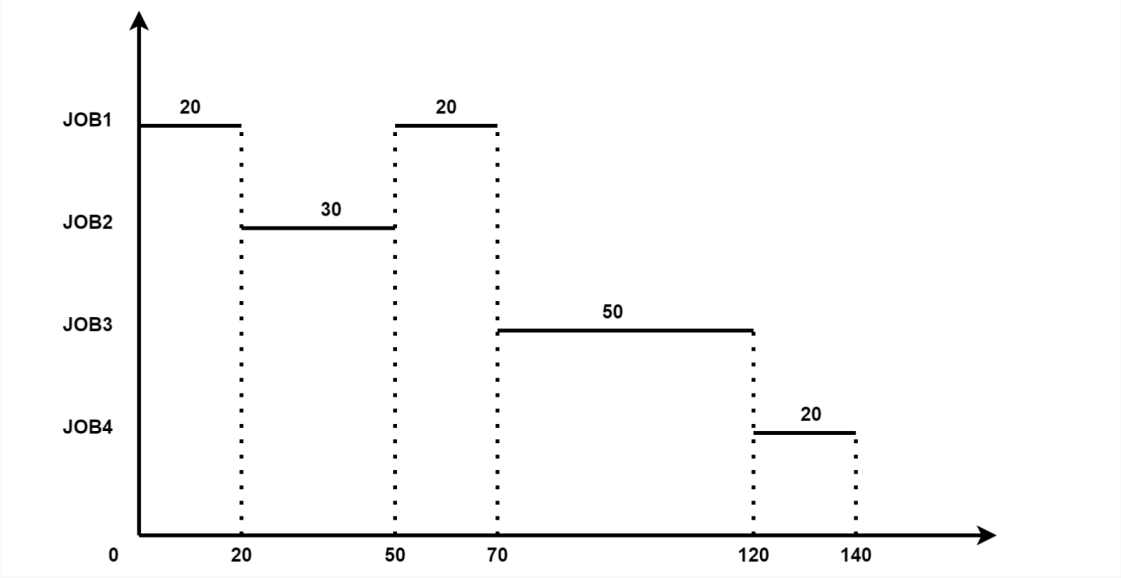
A. 70

B. 75

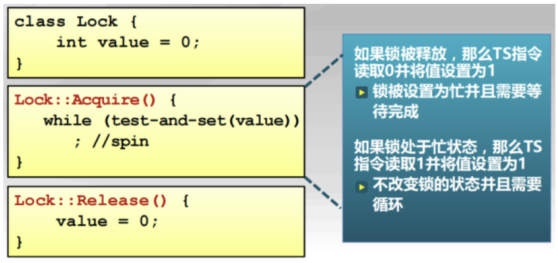
C. 80

D. 85

答案：A。【解析】JOB1进入内存时间为10:00，运行结束时间为11:10，周转时间 为70分钟。JOB2进入内存时间为10:20，运行结束时间为10:50，周转时间为30 分钟。JOB3进入内存时间为11:10，运行结束时间为12:00，周转时间为90分钟。JOB4进入内存时间为10:50，运行结束时间为12:20，周转时间为90 分钟。平均周转时间为：(70+30+90+90)/4=70分钟。

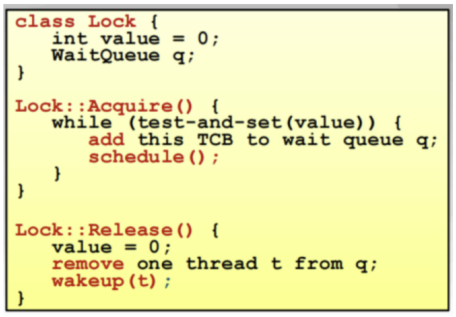


27. 互斥锁是一种用于解决临界区互斥问题的常用手段，对 `acquire()获取锁` 或 `release()释放锁` 的调用必须原子地执行，因此，互斥锁通常采用硬件原语（如 `Test&Set` 指令或 `exchange` 交换指令来实现），根据不同的实现方法，可以实现以下两种锁：



自旋锁

利用无限循环等待阻止程序进入机制实现的锁，具体来说，如上面的示例伪代码所示，当进程申请自旋锁失败时，会一直循环尝试，直到时间片用完被剥夺处理机或者互斥锁被释放；



阻塞锁

申请锁失败的进程会进入阻塞队列，当占有锁的进程释放锁后，会从阻塞队列中唤醒一个被该锁阻塞的进程

以下说法错误的是（）

A. 使用原子操作指令实现的互斥锁适用于单处理器或共享内存多处理器中的多进程同步。

B. 自旋锁不能实现让权等待，但是阻塞锁可以

C. 自旋锁可以实现进程对临界区的先进先出（即按申请顺序访问临界区），不会导致进程饥饿

D. 互斥锁可用于多进（线）程之间，但是只能由获取锁的进（线）程对它进行解锁（即释放）

答案：C。【解析】本题应该选择**C选项**，这道题可以用来学习互斥锁这个考点，我会顺便扩展一些知识，希望大家有所收获；

A正确，互斥锁其实就是一个普通的变量，可以是bool类型也可以是int类型（用0表示锁释放，1表示已上锁），上面这种实现出来的互斥锁，它是支持多临界区的互斥访问。如果有多个临界资源需要互斥访问，我们可以对它们分别实现一个互斥锁，即每把锁一个变量即可；注意，互斥锁变量位于主存里的共享变量，不是某个进程私有的，否则实现不了一个进程拿到锁，别的进程进不去这种效果，因此我们说使用原子操作指令实现的互斥锁适用于单处理器或共享内存多处理器中任意数量的进程同步，因此A选项正确。

B正确，我们可以根据这两把锁的实现方式来看，**自旋锁**在拿不到锁的时候，并没有及时释放处理机，而是一直while循环尝试是否可以获取锁，这个过程一直占有处理机，因此**不符合让权等待**；**阻塞锁**在拿不到锁的时候，就将该进程加入**阻塞队列里**面，因此，**阻塞锁可以实现让权等待**，B 选项正确。

C错误，由于自旋锁具有随机性，没有队列这种数据结构保证进程按照先来后到的顺序使用CPU资源，因此顺序可能会乱，而**阻塞锁**可以实现进程对临界区的先进先出（即按申请顺序访问临界区）。另外如果当某个进程离开临界区时有多个进程等待进入临界区，就有可能导致某些进程一直拿不到锁，产生饥饿现象，因此C选项错误。

D正确，锁和释放锁的必须是同一个进程，否则可能会出现A进程拿到锁之后，但是B进程却释放了这把锁，这时候C获取了这把锁，这样就不满足互斥进入临界区这个规则了，因此在拿到锁和释放锁的时候，需要验证一下身份，例如当某个锁是空闲的时候，A拿到了这把锁，这时候可以把A的进程pid记下来，之后释放锁的时候，验证一下pid是否相等，是的话才可以释放锁，这样就可以保证互斥访问临界区了，因此D正确；

扩展一下，以上这种使用原子操作实现的互斥锁还有可能导致**死锁**的发生，比如在自旋锁实现方式下，某个低优先级进程拿到了锁，在使用这把锁的时候，来了一个高优先级的进程获得处理机，并且它也想进入临界区，一直执行while循环，而低优先级进程又迟迟得不到处理器资源，这时就发生了死锁。

28. 某进程的段表内容如下所示

(段号)	段长	权限	存在位	内存起始地址
82	200	只读	0	6000
...	...	...	...	...
129	100	读写	1	4000
130	304	只读	1	2000

访问并修改$<$ 段号$:$ 段内地址 $>=<082H:130H>$的逻辑地址的数据时，进行地址转换的结果是（）

A. 段缺失异常

B. 越权异常

C. 越界异常

D. 得到内存地址2304

答案：C。【解析】$082H$转成10进制是130，此时查段表，存在位是1，段长是300，而此时段内地址为130H，转成十进制为256*1+3 * 16 = 304，段内地址从0开始的，304是第305号单元，305>304发生越界异常。思考：若把段内地址130H改为12FH，答案是什么？（越权）

总结：

- 要分清十进制和16进制，页表和段表中的页号和段号往往是连续排列，所以是十进制，题目中可能会给16进制，要注意转换，进制统一后，需要考虑段（页）表长度，看段（页）号是否超出段表长度，超出发生**越界异常。**
- 确定好段（页）表项，看存在位是1还是0，若是0，可知段（页）不在内存，发生**段缺失异常。**
- 若是段式，需要考虑段长，同样需要考虑16进制和10进制的转换，以及下标问题。举例：若段长为100，那么段内地址是0-99，100即发生**越界异常。**
- 以上都正常，最后还需要查看权限信息，本题中访问并修改，那么涉及写读和写权限，只读权限的话会发生**越权异常。**

29. 以下关于打开文件操作的说法中，错误的是（）

A. 各进程打开文件的初始读写位置被记录在用户级打开文件表中

B. 若一开始文件F被进程P 1 和P2打开并访问，然后进程P 1 关闭F，则此时在系统打开文件表中，该文件对应的表项的链接计数 `count=1`

C. 一个文件被用户进程首次打开会把文件的FCB或inode调入内存，而不会把文件内容读到内存中

D. 子进程可以访问父进程可以访问的文件

答案：B。【解析】

选项A，文件指针用来记录各进程对文件的读写位置被记录在用户级打开文件表的表项，因此A正确；

选项B错误，链接计数用于硬链接共享文件计数，这里应该是访问（打开）计数；

选项C，正确，１４年真题考过，不再赘述。选项D正确，由于进程拥有进程打开文件表，因此打开文件可以作为进程拥有的资源，同一进程的不同线程共享打开文件表，因此每一个线程都可以用相同的文件描述符ｆｄ访问相同的文件，同样的，子进程可以继承父进程的打开文件表，因此子进程可以访问父进程可以访问的文件，选项D正确。

30. 用户A创建文件F1，用户B在自己目录下创建F1的硬链接F2，并建立F1的软链接F3。用户C创建F3的硬链接F4，再创建F3的软链接F5。现删除文件F1，下列说法正确的是（）

I. F2的计数为1　II. F3的计数为2　III. F4的计数为1　IV.F5的计数为1

V. 文件的现在所有者是用户B

VI. F3可以正常打开文件

VII. F2不能跨文件系统使用，F3可以跨文件系统使用

A. I、II、III、VII　　B. I、III、IV、V

C. I、V、VI　　D. I、II、IV、VII

答案：D 。【解析】硬链接和原文件共享一个inode，而软链接是一个新的link文件，有自己inode。因此，本题中一共有三个inode：文件F1的inode1、文件F1的软链接F3的inode2，软链接F3的软链接F5的inode3；

inode1由F1，F2共享，原计数值为2，现删除F1，计数值-1，变成1，故I正确；

F4是F3的硬链接，故F3和F4共用inode2，所以F3、F4的计数值都是2，II正确、III错误。

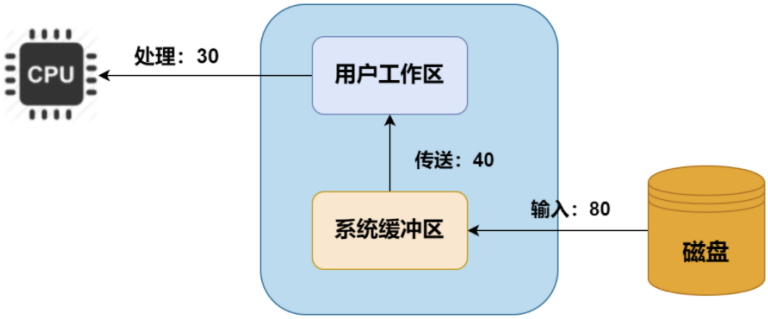
F5的inode3没有其他硬链接，所以计数值为1；文件的所有者owner即使在创建者删除该文件后，只要该inode的计数值不为0，owner都是创建者，因此用户A删除后，inode1的现在所有者还是用户A，V错误；

F3是F1的软链接，保存的是F1的地址，当F1删除后，在用户A的对应目录下的目录项被删除，F3自然找不到对应文件，故不能正常打开文件，VI错误；

最后，有的文件系统不支持inode，如FAT系统，并且就算是两个支持inode结构的文件系统之间，硬链接也不能跨文件系统使用，因为每个文件系统各自维护自己inode数据结构和列表，故F2不能跨文件系统使用，软链接存的是文件路径，是可以跨文件使用的，VII错误。

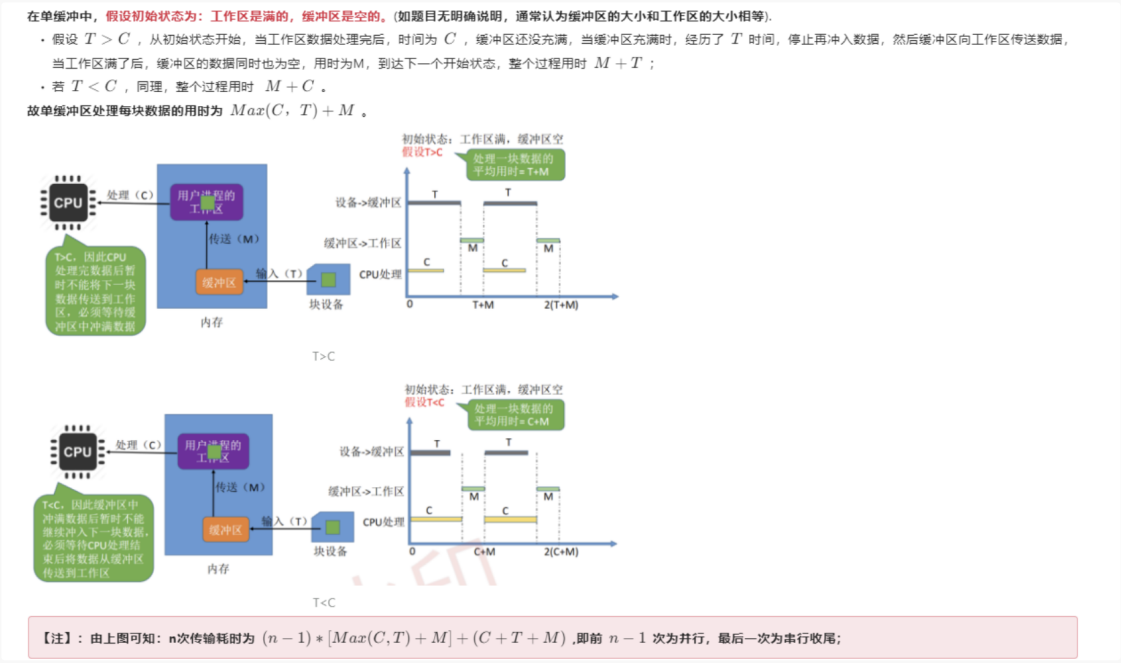
故正确选项为 I、II、IV、VII，答案选D。

31. 设系统缓冲区和用户工作区均采用单缓冲，从磁盘读入一个数据块到系统缓冲区的时间为80μs，从系统缓冲区读入一个数据块到用户工作区的时间为40μs，CPU对用户工作区的一个数据块处理分析时间为30μs，如果有很多数据需要处理，采用单缓冲区传送磁盘数据，则处理一块数据所用平均时间接近（）μs。



A. 120　B. 110　C. 150　D. 70

答案：A。【解析】



32. 下列不属于设备独立性软件的功能的是（）

A. 设备分配及回收

B. 逻辑设备和物理设备的映射

C. 设备初始化

D. 缓冲管理

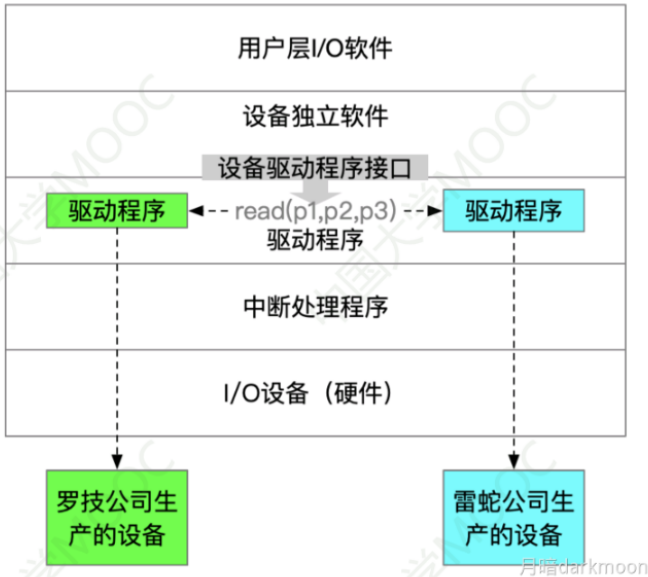
答案：C。设备初始化是由驱动程序控制完成的，19年真题22题选项Ⅰ以及22年真题32题B选项都涉及这个知识点。其他选项的功能都由设备独立性软件实现。

设备独立性软件的功能包括：

1. **设备分配与回收**：管理设备的分配和释放，确保设备资源得到合理利用。
2. **逻辑设备与物理设备的映射**：通过逻辑设备名来请求使用某类设备，并在系统实际执行时将其映射为物理设备名。这增加了设备分配的灵活性，并易于实现I/O重定向。
3. **设备保护**：禁止用户直接访问设备，以保护设备的完整性和安全性。
4. **缓冲管理**：管理缓冲区，用于暂存数据，减少直接I/O操作次数，提高数据传输效率。
5. **差错控制**：监控设备的运行状态，捕获和记录错误日志，并采取相应措施修复错误或通知操作系统和用户。
6. **提供统一接口**：向用户层或文件层提供统一接口，使得不同类型的设备在应用程序中具有相同的操作方式。

驱动程序的功能包括：

1. **设备初始化**：负责初始化硬件设备，包括设备的寄存器、端口、中断等设置，确保操作系统可以正确地与设备进行通信。
2. **实现设备独立性软件和设备控制器之间的通信和转换**：接收设备独立性软件发来的命令和参数，**将命令中的抽象I/O请求转换为具体的I/O操作后**传给控制器，例如，将磁盘块号转换为磁盘的盘面、磁道号及扇区号；又将控制器中所记录的设备状态和I/O操作完成情况，及时地反映给请求I/O的进程。
3. **中断处理**：及时响应由设备控制器发来的中断请求，并根据其中断类型，调用相应的中断处理程序进行处理。
4. 检查用户I/O请求的合法性，了解I/O设备的工作状态，**传递**与I/O设备操作有关的**参数**，设置设备的工作方式。



33. （）属于TCP/IP协议族的应用层。

A. 控制报文协议ICMP

B. 地址解析协议ARP

C. 动态主机配置协议DHCP

D. 开放最短路径优先协议OSPF

答案：C。【解析】ICMP、ARP、OSPF属于网络层协议，而DHCP属于应用层，答案选C。

34. 某通信链路的带宽为3kHz，采用4个相位，每个相位具有x种振幅的QAM调制技术。若该链路在无噪声情况下的极限数据传输速率大于信噪比为30dB条件下的极限数据传输速率，则x至少是()。

A.4 B.8 C.16 D.32

答案：B。【解析】$30(dB)=10\times\log_{10}\frac{S}{N}(dB)$ 解得 $\frac{S}{N}=1000$，由奈氏准则（无噪声）和香农公式（有噪声）：$2W(Baud) \quad \geq \quad W\times\log_2(1+\frac{S}{N})$

$2W\times\log_2 4X \quad \geq \quad W\times\log_2(1+1000)$，解得 $X\geq 8$

35. 某局域网采用CSMA/CD协议，主机A和主机B的距离为1km，传播速率为2x 10^5km/s。t=0时，A发送了数据帧，在A数据发送完毕前，B也发送了数据帧，A、B检测到碰撞的时间相差3μs，则A, B发送的数据发生碰撞的时间，A检测到碰撞的时间为()

A. 5μs，9μs

B. 5.5μs，9μs

C. 4μs，8μs

D. 3.5μs，8μs

> 答案：C。【解析】1km/2×10^5km/s=5μs，AB之间传输时间为5μs。由题知，从A发送数据帧开始计时，则B在5μs时检测到碰撞，A、B检测碰撞的时间相差3μs，那么A在8μs检测到碰撞。从而可推B在3μs时发送数据帧。B发送时，A已经传了3μs，还剩2μs，速度相同，相向而行求相遇时间，可求得在4μs时碰撞，因此答案为C。

36. 关于PPP（点对点）协议说法，正确的有（）

Ⅰ. 既支持异步链路，又支持同步链路

Ⅱ. 可以使用同一个PPP协议连接两个不同网络层协议

Ⅲ. 既支持全双工链路，也支持半双工链路

Ⅳ. 实现了数据的检错和纠错

Ⅴ. 不使用帧编号

A. Ⅰ、Ⅱ、Ⅴ

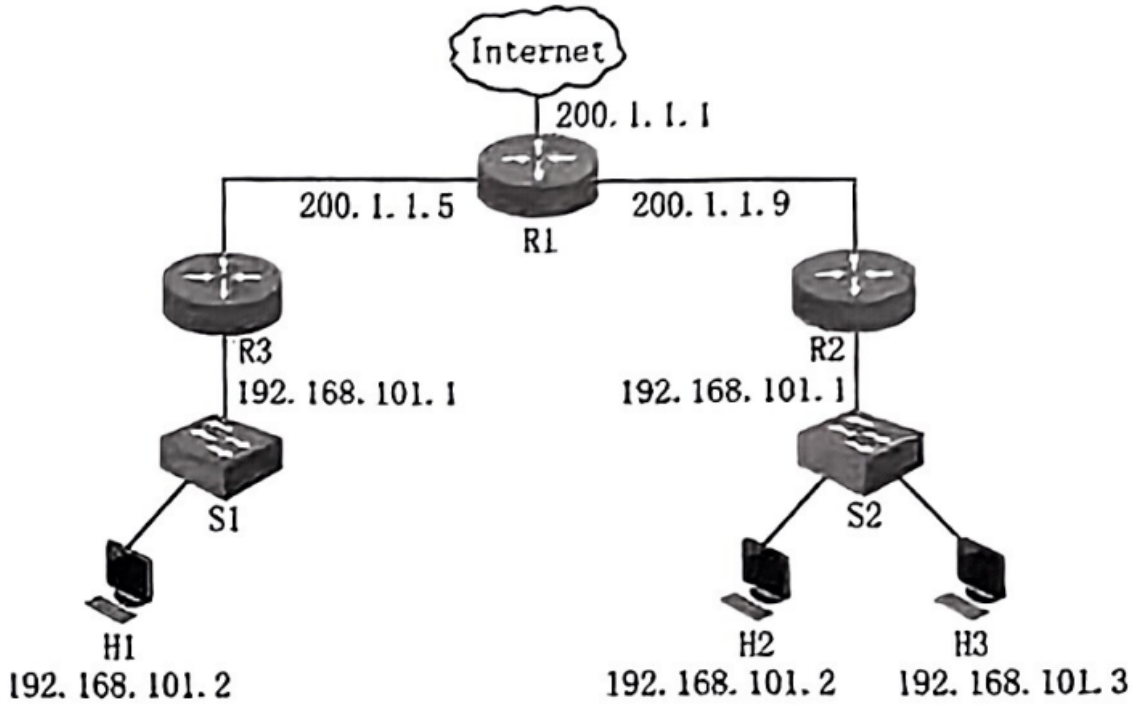
B. Ⅰ、Ⅲ、Ⅴ

C. Ⅰ、Ⅲ、Ⅳ

D. Ⅱ、Ⅲ、Ⅴ

> 答案：A。【解析】Ⅰ正确，PPP协议支持多种类型的链路，包括异步链路和同步链路。这使得PPP协议能够在多种通信环境中灵活应用。
>
> Ⅱ正确，PPP协议的一个重要特性是其网络控制协议（NCP）部分，它允许在同一条物理链路上同时支持多种网络层协议。
>
> Ⅲ错误，PPP协议仅支持全双工通信。
>
> Ⅳ错误，PPP协议实现了数据的检错功能，通过帧检验序列（FCS）字段来确保接收到的数据无差错。然而，PPP协议并不具备纠错能力。当检测到错误时，PPP协议通常会丢弃有差错的帧，并可能请求重传。
>
> Ⅴ正确，PPP协议不使用帧编号来标识每个帧。

37. 某网络的拓扑如下图所示，R1、R2和R3之间的链路的子网掩码均为255.255.255.252，当H1向H3发送一个IP分组时，R3发出的分组的源IP和R2收到的分组的目的IP分别为（ ）。



A. 192.168.101.2，200.1.1.10

B. 200.1.1.6，200.1.1.10

C. 200.1.1.6，192.168.101.3

D. 192.168.101.2，192.168.101.3

答案：B。【解析】外网地址全局有效，内网地址仅内部有效，R3与R2是NAT路由器，对穿越内网外网的IP分组实行地址转换。R3收到的分组的源IP由192.168.101.2转化成R3连接R1的接口的IP地址，由于链路的掩码为255.255.255.252，所以端口的IP为200.1.1.6，同理R2连接R1的端口IP为200.1.1.10，R2收到的分组的目的IP为200.11.10，经过R2进行转换后才变为192.168.101.3，选择B选项。

38. BGP交换网络的可达信息是（）

A. 到达某个网络最小度量的距离以及下一跳路由器

B. 到达某个网络最小跳数的距离以及下一跳路由器

C. 到达某个网络的下一跳路由器

D. 到达某个网络所经过的路径

答案：D。【解析】

RIP，OSPF， BGP 总结

主要特点	RIP	OSPF	BGP
网关协议	内部	内部	外部
路由表内容	目的网络，下一跳，距离	目的网络，下一跳，距离	目的网络，完整路径
最优通路依据	跳数	费用或者其他度量	多种有关装部
算法	距离—向量协议	连路状态协议	路径—向量协议
传送方式	传输层UDP	IP数据报	建立TCP连接
其他	简单、效率低、跳数为16不可达：好消息传得快，坏消息传得慢	效率高、路由器频繁交换信息，难维持一致性：规模大。统一度量为可达性	

39. 在TCP协议里面，主机A发送若干数据给数据B，那么下列说法错误的是（）

I.若主机B收到两个相同的分组，那么直接丢弃晚来的分组即可，无需其他任何处理

II.A收到正确的确认报文后，滑动窗口应该往后（序号大的方向）滑动

III.A的发送缓存一定是发送窗口的子集

IV.B的接收窗口一定是接收缓存的子集

A.II、III

B.I、II、III

C.I、III、IV

D.I、II、III、IV

答案：B。【解析】主机B收到两个相同的分组，一定是B的确定报文段丢失了，应该丢失晚到的报文段，然后再发一个确认，I错误。若接收窗口变小，滑动窗口可能不变。总结如下：

- 发送缓存：应用层想要发送的数据。缓存空间有限并且循环使用，逻辑上是一个圆。其中包含两个部分：

  ①发送方的应用程序传送给发送方TCP准备发送的数据。

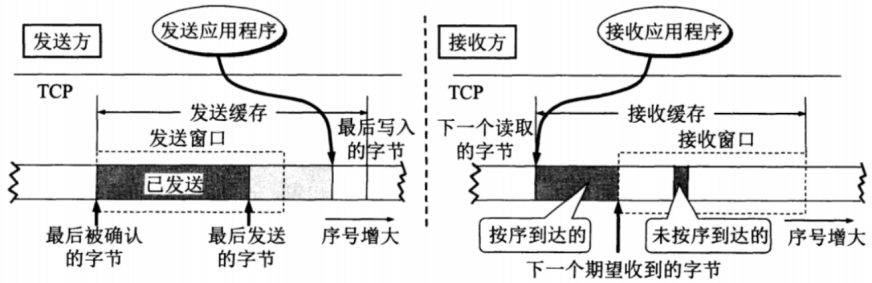
  ②TCP已发送出但未收到确认的数据。

- 发送窗口：发送窗口里的序号表示允许发送的序号。包含：①已经发送了的但是还未收到确认的②允许发送但是还未发送的。
  - 发送窗口左端点在收到新的确认后往右移动，没收到确认则不动。
  - 发送窗口右端点通常是不断往右移动，因为收到了新的确认。但是也可能不动，因为接收发的接收窗口变小了，导致不动。但是TCP标准不赞成往左移动，因为窗口里的数据大部分已经发送出去，现在又要收缩窗口，不让发送这些数据，这样就会产生一些错误。

- 接收缓存：即将交给应用层的数据。包含：①按序到达的、但尚未交付应用进程的数据②未按序到达的

- 接收窗口：用来接收即将到达的数据，按序达到的数据，发送确认后接收窗口往右移动，但是不能超过接收缓存的范围。

> 发送窗口的序号集是发送缓存序号集的子集。
>
> 接收窗口的序号集是接收缓存序号集的子集。



(a) 发送缓存和发送窗口　　(b) 接收缓存和接收窗口

40. 主机H登录到FTP服务器后进行上传文件，若H向服务器上传一个大小为34000B的文件F。约定最大段长为2000B，且均以MSS段向服务器发送数据，设接收缓存足够大，拥塞控制初始阈值为8000B，忽略TCP的传输延迟，传输过程中未发生差错、丢包、乱序的现象，那么从开始建立数据连接开始，到确认F已被服务器全部接收为止，至少需要（）RTT。

A. 4.5

B. 5.5

C. 6

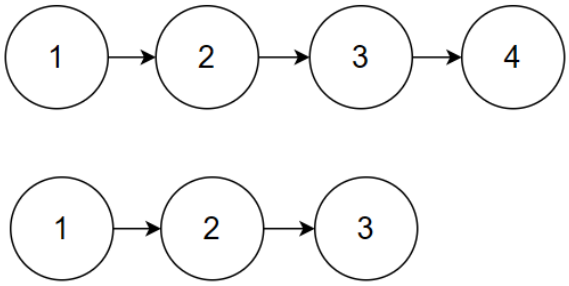
D. 7

> 答案：C。【解析】从建立数据连接开始，就是从TCP建立连接开始计时，到确认F已被服务器接收，那么计时应该到最后一个报文的确认被主机H收到为止。这种题最好转化成MSS来计算，MSS为2000B，那么文件大小为17MSS，拥塞控制的初始阈值为4MSS。
>
> 第1个RTT，第一、二次TCP握手
>
> 慢开始阶段：
>
> 第2个RTT，发送1MSS数据，拥塞窗口增大到2MSS（传输共计1MSS）
>
> 第3个RTT，发送2MSS数据，拥塞窗口增大到4MSS（传输共计3MSS）
>
> 拥塞避免阶段：
>
> 第4个RTT，发送4MSS数据，拥塞窗口增大到5MSS（传输共计7MSS）
>
> 第5个RTT，发送5MSS数据，拥塞窗口增大到6MSS（传输共计12MSS）
>
> 第6个RTT，发送5MSS数据，拥塞窗口增大到7MSS（传输共计17MSS，传输完毕）
>
> 故答案选C。
>
> 注意：有部分同学没注意MSS大小，拥塞避免阶段第一次窗口变化应该是8000B→10000B，如果是8000B→9000B，答案可能误选7rtt

二、综合应用题：41 ~ 47小题，共70分。

41. （13分）（13分）大数加法是能够对超长位数的数字进行相加，假如用不带头节点的单链表表示一个十进制数字，请设计一个空间复杂度尽可能高效的算法：LinkNode* bignumAdd(LinkNode* L1, LinkNode* L2)，实现两个大数的加法，返回结果大数的表示链表首元结点指针。

示例：



结果：4321+321=4642。(十进制)

要求：

（1）给出算法的基本设计思想

（2）根据设计思想，采用C或C++语言描述算法，关键之处给出注释。

首先题意是小端存储，下列方法均有错误：

- 看成大端，三次reverse，最多给3分
- 简单的把结点值相加，比如567+234（小端），是不是得到7 9 （11）？没考虑进位要扣分，扣分不超过3分
- 没考虑最后的高位进位，即没添加新节点，要扣分，扣分2分
- 如果新增一个链表存储结果，扣1分，不符合题目要求的最小空间复杂度（参考答案给的就是开辟新的链表，要扣分）
- 如果转换成数组，进行计算，再转回链表，空间复杂度On，扣3分
- 如果把大数转成longlong或其他类型，进行加法，再存进去，因为大数加法本身就是为了防止超出基本类型数据范围而设计的，扣9分，

（1）算法设计思想：

①从链表头部（十进制低位）进行加法，sum为当前节点的和，超过10的话，同步修改L1和L2的当前结点的值为sum%10，用t存放进位信息

②当某个链表遍历结束后，应继续使用较长的链表存放结果，实现O（1）空间复杂度，运算时要加上低位的进位

③若两个链表都遍历完，仍有进位，需要在较长链表的末尾新增一个结点保存进位

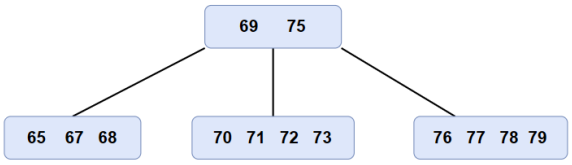
④返回较长链表的首元结点的指针

(2)

```
ListNode* bignumAdd(ListNode* l1, ListNode* l2) {
        ListNode *p = l1, *q = l2, *pre_p = NULL, *pre_q = NULL; //pre_p和pre_q记录p,
q的前驱，使最后停留在尾结点，方便最后的进位的插入结点操作
        int t = 0; //记录进位信息
        int len1 = 0, len2 = 0; //记录l1和l2的链表长度
        while (p || q){
            int sum = 0, p_val= 0, q_val = 0;     //计算当前位数的和
            if (p) p_val = p->val;
            if (q) q_val = q->val;
            sum = p_val + q_val + t;
            if (p) p->val = sum % 10;
            if (q) q->val = sum % 10;
            t = sum / 10;
            if (p) {
                pre_p = p;
                p = p->next;
                len1 ++ ;
            }
            if (q) {
                pre_q = q;
```

```
                q = q->next;
                len2 ++ ;
            }
        }
        if (t) {    //处理最后的进位
            if (len1 > len2) pre_p->next = new ListNode(t);
            else pre_q ->next = new ListNode(t);
        }
        return len1 > len2 ? l1 : l2;
```

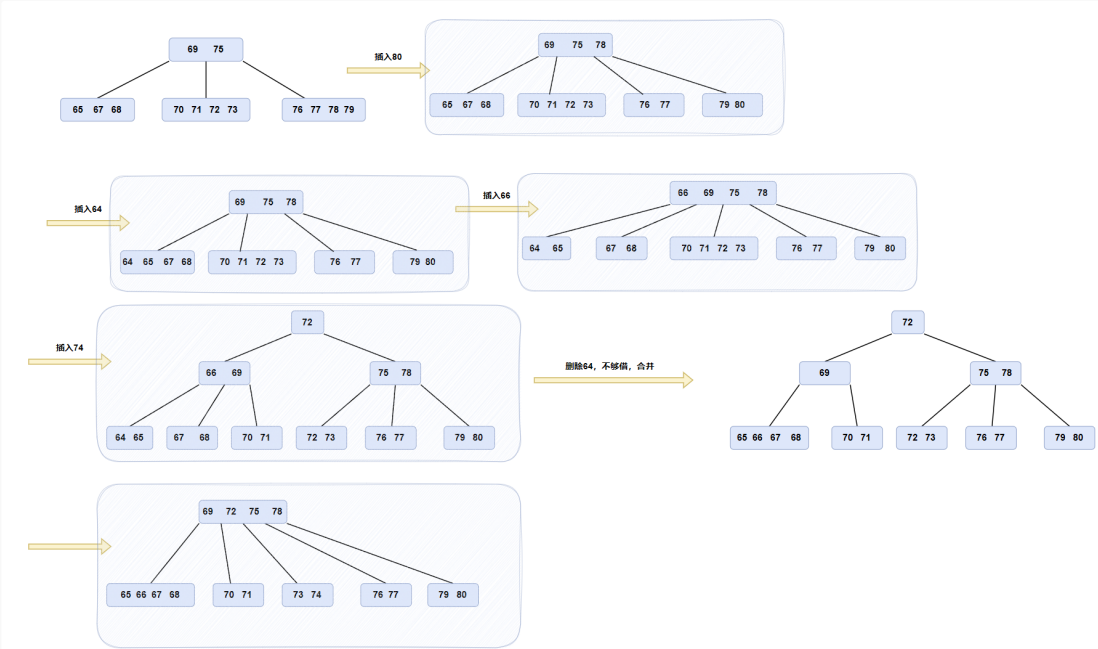
42. （10分）如图所示是一棵5阶B树，请回答以下问题：



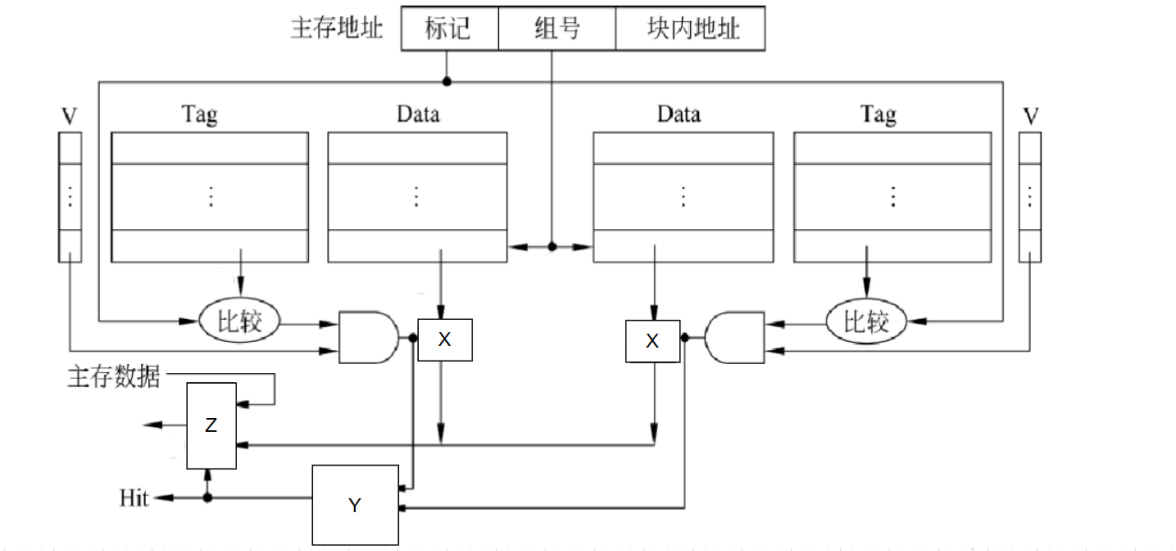
（1）请依次画出插入关键字80，64，66，74的B树形态

（2）请在（1）的基础上画出删除关键字64后B树的形态

【解析】



43. （13分）假设主存地址为26位，按字节编址，指令Cache和数据Cache采用回写（Write Back）写策略和LRU替换算法，主存块大小为$64B$，共8个cache行，（1）、（2）、（3）、（4）题均采用下图所示cache，请回答以下问题：



（1）该cache采用什么映射方式?

（2）假设比较器的输出为Ci，有效位为Vi（i=1，2），试写出Y的输出Hit（Hit=1表示cache命中）的逻辑运算表达式。

（3）图中X用来控制数据区某行数据的输出，那么X应该是什么？Z是什么？Z的功能是什么？

（4）有如下C语言程序段：

```
for (k = 1; k < 64; k ++ ){
    A[k] += A[k - 1];
}
```

若数组A为double类型，double型数据占8B，变量k为int型，int型数据占4B，变量k分配在寄存器中，数组A在主存中的起始地址为080 00C0H，则该程序段执过程中，访问数组A的数据Cache缺失率为多少？（分数表示）

（5）若cache总容量为$535B$，则此时cache又采用什么映射方式？该cache中比较器的位数是多少?

【答案】（1）二路组相联（组相联不给分）（1分）

（2）$Hit = C_1V_1 + C_2V_2$（2分）

（3）X：三态门（1分），Z：多路选择器（1分）。Z的功能是通过Hit信号控制数据来自Cache还是主存，若Hit为1表示cache命中，CPU从cache中取数据，反之从主存取数据。（2分）

（4）8/189（3分）

（5）4路组相联映射。（3分）19bit。（1分）

【解析】（1）比较器个数为2，因此是二路组相联（组相联不给分）（1分）

（2）Y为或门（异或门也可，后面改成⊕，合理即可），$Hit = C_1V_1 + C_2V_2$，表示只要组内有一行命中，则cache命中。（2分）

（3）X：三态门（1分），Z：多路选择器（1分）。Z的功能是通过Hit信号控制数据来自Cache还是主存，若Hit为1表示cache命中，CPU从cache中取数据，反之从主存取数据。（2分）。

（4）程序段访问数组A[0] ~ A[63]共64个元素，每个double元素8B，所以需要512B，cache刚好装下，只需要分析访问元素总次数和缺失的次数，由程序段可知，`A[k] = A[k] + A[k - 1];`一条指令两次读一次写，读A[0] ~ A[62]共63次，读A[1] ~ A[63]共63次，写A[1] ~ A[63]共63次，一共访存189次。每块中，只有访问第一个元素才发生cache缺失，缺失的元素依次是A[0],A[8]...A[56]共8次，因此缺失率为8/189。(3分，可根据过程酌情给分)

（5）4路组相联，19bit。内存地址空间为64M=2^{26}，物理地址为26位。按字节编址，主存块长64B，块内地址为6bit。cache数据容量8 * 64B=512B，标记阵列容量为535B-512B=23B，一共8行，**每行的标记信息一共23B/8=23bit**，回写方式需要1bit脏位，另外1bit有效位，还要考虑LRU算法位

不妨设采用k路组相联，那么LRU算法位为log_2k位，若计算出①k=1，那便是直接映射②1<k<8，k路组相联③k=8，全相联映射。

因此tag位为$23 - 1 - 1 - log_2k = (21 - log_2k)bit$。26位主存地址由tag位+组号+块内地址组成，因此可以知道组号位数为$26 - (21 - log_2k) - 6 = (log_2k - 1)bit$，可知该cache采用$8/2^{log_2k-1} = 16/k$路组相联

即$k = \frac{16}{k}$,得$k = 4$

即采用4路组相联映射。（3分）

比较器位数为tag的位数，为19bit。（1分）

44. （10分）对于以下指令序列：

```
Loop:
      lw $2, 100($6)     ①内存地址$6 + 100处加载一个字到寄存器$2中
      add $3, $2, $7     ②将寄存器$2和$7中的值相加，并将结果存储在寄存器$3中
      sw $3, 100($6)     ③将寄存器$3中的值存储回内存地址$6 + 100处
      lw $5, 200($6)     ④从内存地址$6 + 200处加载一个字到寄存器$5中
      sub $4, $5, $2     ⑤从寄存器$5中的值减去寄存器$2中的值，并将结果存储在寄存器$4中
      sw $4, 300($6)     ⑥将寄存器$4中的值存储回内存地址$6 + 300处
      addi $6, $6, 8     ⑦将寄存器$6中的值增加8
      bne $6, $8, Loop   ⑧检查寄存器$6和$8中的值是否不相等。如果不相等，跳转到Loop处执行，相等则继续执行下一条指令（跳出循环）
```

假设在采用"转发"技术的5段流水线中执行上述循环50次，该流水线数据通路中，主要功能单元的操作时间为：存储器—200ps，ALU和加法器—100ps，寄存器堆（读或写）—50ps。寄存器写口和寄存器读口分别被安排在一个时钟周期的前、后半个周期内独立工作，分支指令的分支延迟损失时间片为3（需要3个条nop指令阻塞）。**假设执行循环时流水线已经进入正常状态，即计算时按第一条指令在第1个时钟周期完成**，要求回答下列问题：

（1）在不考虑流水段寄存器、多路选择器、控制单元、PC、扩展器和线路等延迟的情况下，5段流水线处理器的最小时钟周期为多少?

（2）不采用分支预测时,那么每次循环执行需要多少时钟周期?

（3）采用分支预测，并每次预测转移（即预测成功无延迟时间片损失，预测失败仍需要3个分支延迟时间片损失），则50次循环里每次循环执行需要多少时钟周期?

（4）load-use数据冒险可以通过调整指令减少冒险带来的阻塞，题目所示代码应将④号指令调整到第几条指令之后最佳？请给出调整后并采用静态分支预测并总是预测转移设计下的程序执行时间（时钟周期数）。

【答案】（1）200ps（1分）

（2）13（1分）

（3）前49次循环每次需要10个时钟周期，最后一次13个时钟周期。

（4）①；403个时钟周期。

【解析】(1)200ps。因为最复杂的部件(存储器)所用的最长时间为200ps,所以,在不考虑流水线寄存器、多路选择器、控制单元、PC、扩展器和线路等延迟的情况下，5段流水线处理器的最小时钟周期为 200ps。（1分）

（2）因为循环中有两个 load-use 冒险和一个控制(分支)冒险,每个 load-use 冒险在采用转发技术下需要阻塞一个时钟周期。而分支指令的分支延迟损失时间片为3，所以共有 2+3=5次阻塞，一共8条指令，因此每次循环的执行共需 8+5=13 个时钟周期。（2分）

（3）若采用静态分支预测，并总是预测转移,则前面49次循环中的分支预测都能成功，因而无需额外的时钟周期,故一次循环需 8+2=10个时钟周期（2分）,而最后1次循环中的分支预测不成功,需要在流水线中冲刷掉3条指令,故最后一次循环需8+2+3=13 个时钟周期。（1分）

（4）可以将无关指令插人到 load-use 数据相关指令之间，即调整指令④到指令①后，指令序列如下：（2分）

```
Loop:  lw    $2, 100($6)
       lw    $5, 200($6)
       add   $3, $2, $7
       sw    $3, 100($6)
       sub   $4, $5, $2
       sw    $4, 300($6)
       addi  $6, $6, 8
       bne   $6, $8, Loop
```

指令顺序调整后,消除了 load-use 数据冒险，在采用静态预测(总是预测转移)的情况下，前49次每次循环需8个时钟周期，最后1次预测错误,需在流水线中冲刷3条指令,因而需 8+3=11 个时钟周期。共需 49×8+1×11=403 个时钟周期。（2分）

45. （9分）在页式虚拟存储器中，一个程序由1-8共8个页面组成，程序执行过程中依次访问的页面序列为：3，4，2，6，4，3，7，4，3，6，3，4，8，4，6；采用固定分配局部置换策略，为进程分配3个页框，回答：

（1）在先进先出（FIFO）算法和最近最少使用（LRU）算法下，缺页率各是多少？（分数表示）

（2）若采用时钟置换（CLOCK）算法，缺页次数是多少？若采用改进的CLOCK算法，需要在页表项中增加什么信息？相比CLOCK算法，他带来的优点是什么？

【答案】（1）FIFO缺页率为12/15 = 4/5（2分）；LRU缺页率为9/15=3/5。（2分）

（2）缺页9次；（2分）

（3）增加修改位（1分）；在访问位同为0的情况下优先置换修改位为0的页面，因为未修改过的页面可以直接淘汰而不用写回磁盘，尽管增加了扫描次数（在内存），但是**减少了I/O次数。**（2分，言之有理即可）

【解析】（1）FIFO：3，4，2，6，4，3，7，4，3，6，3，4，8，4，6，缺页率为10/15 = 2/3（2分）

引用串P	3	4	2	6	4	3	7	4	3	6	3	4	8	4	6
内存	3	4	2	6	6	3	7	4	4	6	3	3	8	4	6
		3	4	2	2	6	3	7	7	4	6	6	3	8	4
			3	4	4	2	6	3	3	7	4	4	6	3	8
是否缺页	√	√	√	√		√	√	√		√	√		√	√	√

LRU：3，4，2，6，4，3，7，4，3，6，3，4，8，4，6；缺页率为9/15=3/5（2分）

引用串P	3	4	2	6	4	3	7	4	3	6	3	4	8	4	6
内存	3	4	2	6	4	3	7	4	3	6	3	4	8	4	6
		3	4	2	6	4	3	7	4	3	6	3	4	8	4
			3	4	2	6	4	3	7	4	4	6	3	3	8
是否缺页	√	√	√	√		√	√			√			√		√

（2）CLOCK：3，4，2，6，4，3，7，4，3，6，3，4，8，4，6；缺页9次；（2分）

CLOCK 算法的缺页情况：

引用串P	3	4	2	6	4	3	7	4	3	6	3	4	8	4	6
内存	3	3	3←	6	6	6←	6	4	4	4	4	4	4←	4←	4←
	←	4	4	4←	4←	4	7	7←	7←	6	6	6	6	6	6
		←	2	2	2	3	3←	3	3	3←	3←	3←	8	8	8
是否缺页	√	√	√	√		√	√	√		√			√		

注：←表示指针位置。

增加访问位和修改位（2分）；在访问位同为0的情况下优先置换修改位为0的页面，因为未修改过的页面可以直接淘汰而不用写回磁盘，尽管增加了扫描次数（在内存），但是**减少了I/O次数。**（1分，言之有理即可）

46. （6分）存放在某个磁盘上的文件系统采用混合索引分配方式，其FCB中共有13个地址项，10个直接地址，一次、二次、三次间接地址各一个。假设每个盘块的大小为512B，若盘块号需要占4B，则：

（1）该文件系统允许文件的最大长度是多少？（给出计算式即可）

（2）一个文件的大小为8MB，该文件的FCB里用了多少个地址项？

（3）假设某个文件的FCB已在内存，但其它信息均在外存，为了修改某个字节的内容，最少需要多少次访问磁盘？最多多少次访问磁盘？

【答案】（1）（10 + 128 + 128 * 128 + 128 * 128 * 128） * 512B（2分）

（2）12个。10个直接地址，1个一次间接地址，1个二次间接地址。（2分）

（3）最少2次，最多5次。（2分）

【解析】（1）每个磁盘块可存放512B/4B=128个盘块，因此该文件系统中一个文件的最大长度可达（10 +128 + 128 * 128 + 128 * 128 * 128） * 512B。

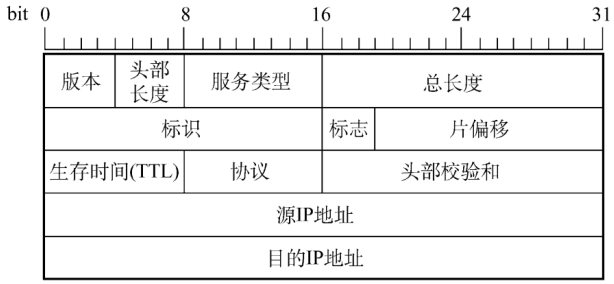
(2) (10 + 128) * 512 / 1024 = 69 KB ＜ 8MB = 8192KB ＜（10 + 128 + 128 * 128）* 512 / 1024 = 8261KB，最多用到二次间接地址，因此10个直接地址，1个一次间接地址，1个二次间接地址。共计12个地址项。

（3）最少2次，即通过直接地址读出磁盘块（1次），修改后写回磁盘（1次）。最多5次，即所访问位置用到了三次间接地址，读出磁盘块（4次），修改后写回磁盘（1次），一共5次。

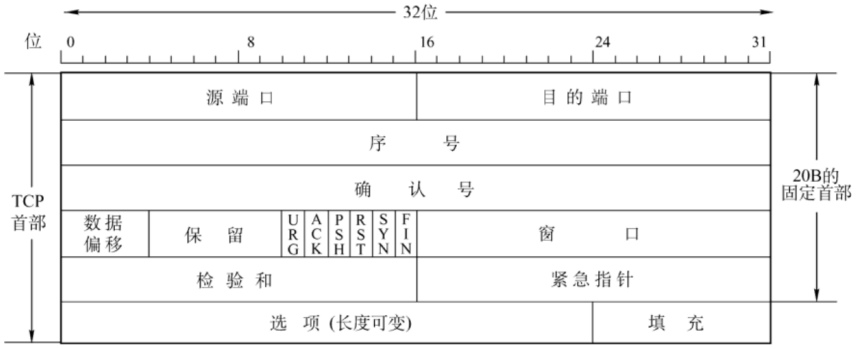
47. （9分）小明在学习408时，对计网非常感兴趣，于是通过抓包软件捕获了若干主机H与服务器S的有关TCP通信的IP分组，由于不小心打翻了墨水，导致部分数据不可见，具体如下表所示，已知主机H的IP 地址为192.168.0.8，服务器S的IP地址为211.68.71.80。小明找到你这位计网高手，请你帮他分析问题并还原数据（用16进制表示）：

编号	IP 分组的前 40 字节内容（十六进制）				
1	45 00 00 30	01 9b 40 00	80 ① 1d e8	c0 a8 00 08	d3 44 47 50
	0b d9 13 88	84 6b 41 c5	②	70 02 43 80	5d b0 00 00
2	45 00 00 30	00 00 40 00	31 ① 6e 83	d3 44 47 50	c0 a8 00 08
	13 88 0b d9	e0 59 9f ef	84 6b 41 c6	70 12 16 d0	37 e1 00 00
3	45 00 00 28	01 9c 40 00	80 ① ③	c0 a8 00 08	d3 44 47 50
	0b d9 13 88	84 6b 41 c6	e0 59 9f f0	50 f0 43 80	2b 32 00 00
4	45 00 00 38	01 9d 40 00	80 ① 1d de	c0 a8 00 08	d3 44 47 50
	0b d9 13 88	④	e0 59 9f f0	50 18 43 80	e6 55 00 00
5	45 00 00 28	68 11 40 00	31 ① 06 7a	d3 44 47 50	c0 a8 00 08
	13 88 0b d9	e0 59 9f f0	84 6b 41 d6	50 10 16 d0	57 d2 00 00

注：IP 分组头和 TCP 段头结构分别如题 47-a 图，题 47-b 图所示。



题 47-a 图　IP 分组头结构



题 47-b 图 TCP 段头结构

（1）表中①、②、④分别是多少?
（2）③是多少?
（3）若此时捕获并给出主机H发出的IP分组中TCP首部如下图所示，请问主机H和服务器S本次TCP连接中主机H发送多少数据？接收多少数据？（十进制表示）若主机之后再也没收到服务器S的数据，并且没有发生任何异常，请问下一个来自服务器S的IP分组中，序号和确认号字段分别是多少?

H发出的IP分组	45 00 00 28　01 b6 40 00　80 ① 1d d5　c0 a8 00 08　d3 44 47 50 0b d9 13 88　84 6b 49 af　e0 59 9f f0　50 11 43 80　23 a7 00 00

【答案】（1）①：06H，②：00 00H，④：84 6B 41 C6H。（各1分）
（2）③：1D EFH。（2分）
（3）主机H发送：2025B（1分），接收0B（1分），序号为E0599FF0H（1分），确认号为846B49B0H（1分）。
【解析】(1)主机和服务器采用TCP通信，故协议字段为6，第一个和第二个IP分组，分别来自H和S，且SYN都为1，那么一定是第一次握手和第二次握手，那么如何分辨谁是第一次谁是第二次呢？由于第一个分组中TCP首部的序号是846b 41c5，第二个的确认号是846b 41c6，那么IP分组1为第一次握手报文，由主机H申请与服务器建立连接。TCP第一次握手，自己的序列号是随机产生的，但是主机H并不知道服务器的初始序号，所以②中的确认号要填全0。 顺着往下整理，第三个IP分组的第四个字节是28H，转为十进制也就是40，刚好为IP首部和TCP首部长度之和，因此第三次握手不带携带任何数据，不携带数据不消耗序号，所以④处数据应为84 6B 41 C6H。

（2）③处对应IP首部校验和，计算规则是IP首部中除了校验和字段之外的所有字段分成16bit的段，依次进行16位二进制求和，高位溢出的一律加到低位（回卷），求得结果进行求反，填入首部校验和字段，求得结果为1D EFH。

（3）主机H从IP分组4开始给服务器S发送数据，IP分组4的序号字段是846b41c6，而现在的IP分组是846b 49af，相差7E9H，转换成10进制是2025，确认字段和IP分组4一致为e059 9ff0，可知服务器没有给主机发送任何数据，并且FIN字段为1，表示主机H断开连接。因为本此TCP连接主机H发送：2025B，接收0B。FIN报文消耗一个序号，服务器给主机发送的下一个报文的确认字段为846B 49AFH + 1H = 846B49B0H，序号和主机发送的确认号一致，为E0599FF0H。